LENGUAJES DE PROGRAMACIÓN I

Universidad Nacional Mayor de San Marcos · Facultad de Ingeniería Electrónica y Eléctrica

Guía de estudio desde cero: C++ para el Examen Parcial

Teoría completa (lenguajes de programación, pseudocódigo, diagramas de flujo y C++) y solucionario detallado de los 10 ejercicios de preparación

Basado en: 10 ejercicios de preparación para el Examen Parcial (semanas 1-7)

Lenguaje: C++

Nivel: Desde cero: no se asume conocimiento previo

Incluye: Pseudocódigo, diagramas de flujo, código probado, ejecuciones reales y pruebas de escritorio

- Algoritmo, código fuente y ejecutable
- Lenguajes de programación y compilación
- Análisis Entrada - Proceso - Salida
- Pseudocódigo
- Diagramas de flujo
- Prueba de escritorio
- Variables y tipos de datos
- cin, getline y cout
- Operadores y precedencia
- División entera y static_cast
- Formato con iomanip
- string: find, substr, replace
- UML: clases y objetos
- struct y class

Material de estudio no oficial elaborado a partir de la hoja de ejercicios del curso. Todos los programas fueron compilados y ejecutados; las salidas mostradas son reales.

Índice

Cómo usar esta guía 2

- Qué encontrarás en cada ejercicio resuelto 2
- Mapa de la guía 2
- Los 10 ejercicios de un vistazo 3
- Cómo estudiar con esta guía 3
- Dónde practicar (gratis) 3
- Archivos de esta carpeta 3
- Convenciones usadas 4

Parte I — Fundamentos: computadoras, algoritmos y lenguajes 5

- 1. ¿Qué es una computadora y qué es un programa? 5
- 2. Algoritmo, código fuente y programa ejecutable 5
- 3. Lenguajes de programación 7
- 4. Metodología para resolver un problema 9
- 5. Pseudocódigo 10
- 6. Diagramas de flujo 12
- 7. Prueba de escritorio 14
- 8. UML básico: el diagrama de clases 14

Parte II — C++ desde cero 16

- 9. Tu primer programa en C++ 16
- 10. Variables y tipos de datos 17
- 11. Entrada y salida de datos 20
- 12. Operadores y expresiones 22
- 13. Conversión de tipos 24
- 14. Formato de salida con `<iomanip>` 25
- 15. Cadenas de caracteres: `string` 26
- 16. Agrupar datos: `struct` y `class` 28

Parte III — Los 10 ejercicios resueltos paso a paso 30

- Ejercicio 1 — Tipos de datos y variables 31
- Ejercicio 2 — Entrada, proceso y salida 34
- Ejercicio 3 — Trazado de expresiones 38
- Ejercicio 4 — Precedencia de operadores 41
- Ejercicio 5 — Debugging de entrada con string 45
- Ejercicio 6 — Procesamiento de código con substr() 51
- Ejercicio 7 — Búsqueda y reemplazo en strings 55
- Ejercicio 8 — Problema comercial integrado 59
- Ejercicio 9 — UML básico + C++ 63
- Ejercicio 10 — Caso integrador de preparación 68

Parte IV — Repaso final 75

- 17. Checklist antes del parcial (respondido) 75
- 18. Los 15 errores más frecuentes 76
- 19. Hoja resumen 76
- 20. Autoevaluación 78
- 21. Glosario 80

Cómo usar esta guía

Esta guía acompaña a la hoja **«10 ejercicios de preparación para el Examen Parcial»** del curso **Lenguajes de Programación I** (semanas 1 a 7, lenguaje C++).

Está escrita para alguien que **empieza desde cero**: no se da por sabido qué es una variable, un compilador, un pseudocódigo o un diagrama de flujo. Primero se explica la teoría con ejemplos sencillos y después se resuelve **cada ejercicio paso a paso**.

Qué encontrarás en cada ejercicio resuelto

Cada uno de los 10 ejercicios trae, cuando corresponde:

1. **Qué se evalúa**: los conceptos que el profesor quiere ver.
2. **Análisis Entrada – Proceso – Salida (E-P-S)** y la elección justificada de los tipos de datos.
3. **Pseudocódigo**.
4. **Diagrama de flujo**.
5. **Código C++ completo y comentado** (todos fueron compilados y probados).
6. **Ejecución real** del programa (lo que aparece en la pantalla).
7. **Prueba de escritorio**: seguir el programa «a mano», instrucción por instrucción.
8. **Errores frecuentes y variantes** que pueden aparecer en el examen.

Mapa de la guía

Parte	Contenido	Para qué sirve
I. Fundamentos	Computadora, programa, algoritmo, lenguajes de programación, compilación, E-P-S, pseudocódigo, diagramas de flujo, prueba de escritorio, UML	Entender las ideas que el examen da por sabidas
II. C++ desde cero	Estructura de un programa, variables y tipos, `cin` / `cout` / `getline`, operadores, precedencia, conversión de tipos, formato de salida, `string`, `struct` y `class`	Tener todas las herramientas para programar
III. Ejercicios resueltos	Los 10 ejercicios de la hoja, uno por uno	Practicar con soluciones completas
IV. Repaso final	Checklist respondido, errores frecuentes, hoja resumen, autoevaluación con respuestas y glosario	Repasar rápido antes del examen

Los 10 ejercicios de un vistazo

N.°	Tema	Nivel	Idea clave
1	Tipos de datos y variables	Básico	`string`, `int`, `double`, `bool`, `char`
2	Entrada, proceso y salida	Básico	Estructura de cualquier programa
3	Trazado de expresiones	Medio	División entera y `static_cast`
4	Precedencia de operadores	Medio	`* / %` antes que `+ -`
5	Debugging de entrada con `string`	Medio	`cin >>` vs. `getline()`
6	Procesamiento de código con `substr()`	Medio	Índices desde 0
7	Búsqueda y reemplazo en strings	Medio-alto	`length()`, `find()`, `substr()`, `replace()`
8	Problema comercial integrado	Medio-alto	Fórmulas + 2 decimales
9	UML básico + C++	Avanzado	Clase, atributos y operaciones
10	Caso integrador	Avanzado	Todo junto, sin `if` ni bucles

Cómo estudiar con esta guía

Si tienes...	Haz esto
Varios días	Lee las Partes I y II completas. Luego intenta cada ejercicio **tú solo, en papel**, y recién después compáralo con la solución.
Un solo día	Lee la Parte II (secciones 9 a 15) y resuelve los ejercicios 2, 3, 4, 5, 7 y 10.
Una hora	Lee la *Hoja resumen* y la lista de *Errores frecuentes* de la Parte IV, y repasa los ejercicios 3, 5 y 10.

> **Consejo**
>
> Estudiar programación es como aprender a nadar: leer ayuda, pero **lo que de verdad funciona es practicar**. Escribe el código a mano (en el examen probablemente será en papel), luego cópialo en la computadora y compruébalo.

Dónde practicar (gratis)

- **Sin instalar nada**: compiladores en línea como *OnlineGDB* (onlinegdb.com, elige el lenguaje C++) o *Programiz* (programiz.com/cpp-programming/online-compiler).
- **Instalado en tu computadora**: *Dev-C++* (versión de Embarcadero), *Code::Blocks* (la versión que trae MinGW) o *Visual Studio Code* con el compilador MinGW.
- **Para pseudocódigo y diagramas de flujo**: *PSeInt*, que además dibuja el diagrama de flujo a partir de tu pseudocódigo.

Archivos de esta carpeta

- `codigo/`: los programas C++ de cada ejercicio (`ej01_...cpp` hasta `ej10_...cpp`). Se pueden abrir con cualquiera de los programas de arriba.

- `diagramas/` : todos los diagramas de flujo y figuras de la guía.
- `Enunciado_original_10_ejercicios.pdf` : la hoja original de ejercicios.

Convenciones usadas

- Lo que se escribe en el código va en `letra monoespaciada`.
- En pseudocódigo, la flecha `←` significa **asignación** («guardar en»): `pago ← horas * tarifa`.
- En los **archivos de código** evitamos tildes y la letra ñ dentro del texto que se muestra (escribimos `Anio` en lugar de `Año`). No es un error: así el texto se ve bien en cualquier consola, incluida la de Windows, que suele mostrar mal esas letras. El propio profesor lo hace en sus códigos (`"Marron"`, `"numero"`). **En el examen en papel sí puedes escribir con tildes.**
- Los índices de un `string` siempre empiezan en **0**.

Parte I — Fundamentos: computadoras, algoritmos y lenguajes

1. ¿Qué es una computadora y qué es un programa?

Una **computadora** es una máquina que **procesa datos siguiendo instrucciones**. No «piensa»: hace exactamente lo que le indicamos, muy rápido y sin equivocarse en las cuentas. Sus partes principales son:

Parte	Qué hace	Analogía (una cocina)
CPU (procesador)	Ejecuta las instrucciones y hace los cálculos	El cocinero
Memoria RAM	Guarda los datos y el programa *mientras* se ejecuta. Se borra al apagar	La mesa de trabajo
Almacenamiento (disco, SSD)	Guarda los archivos de forma permanente	La alacena
Dispositivos de entrada (teclado, mouse)	Permiten ingresar datos	El cliente que hace el pedido
Dispositivos de salida (pantalla, impresora)	Muestran los resultados	El plato servido

Un **programa** es un conjunto de instrucciones que la computadora sigue, una tras otra, para resolver una tarea: calcular un sueldo, mostrar una nota, procesar un código, etc.

La computadora solo entiende ceros y unos

Por dentro, todo (números, letras, imágenes, programas) se guarda como **bits**. Un bit es la unidad mínima de información y solo puede valer **0 o 1** (como un interruptor apagado o encendido). Ocho bits forman un **byte**. Por ejemplo, la letra `A` se guarda como el byte `01000001`.

Escribir programas directamente en ceros y unos sería casi imposible. Por eso se inventaron los **lenguajes de programación**: nos dejan escribir instrucciones parecidas al lenguaje humano, y luego un programa traductor las convierte a ceros y unos.

2. Algoritmo, código fuente y programa ejecutable

Estos tres conceptos son la primera pregunta del *checklist* del parcial. Es fundamental no confundirlos.

2.1 Algoritmo

Un **algoritmo** es una **secuencia finita, ordenada y precisa de pasos** para resolver un problema.

Sus características son:

- **Preciso**: cada paso dice exactamente qué hacer, sin ambigüedades.
- **Definido (determinista)**: con los mismos datos, siempre da el mismo resultado.
- **Finito**: termina en algún momento.
- Tiene **entrada** (datos), **proceso** (pasos) y **salida** (resultado).

Un algoritmo **no depende de ningún lenguaje de programación**: se puede escribir en español, en pseudocódigo o como diagrama de flujo.

Ejemplo cotidiano — *preparar un vaso de limonada*:

1. Exprimir 2 limones en un vaso.
2. Agregar 2 cucharadas de azúcar.
3. Llenar el vaso con agua.
4. Mover hasta disolver el azúcar.
5. Servir.

Ejemplo de computación — *calcular el promedio de 3 notas*:

1. Pedir las tres notas: `n1`, `n2`, `n3`.
2. Calcular `promedio = (n1 + n2 + n3) / 3`.
3. Mostrar el promedio.

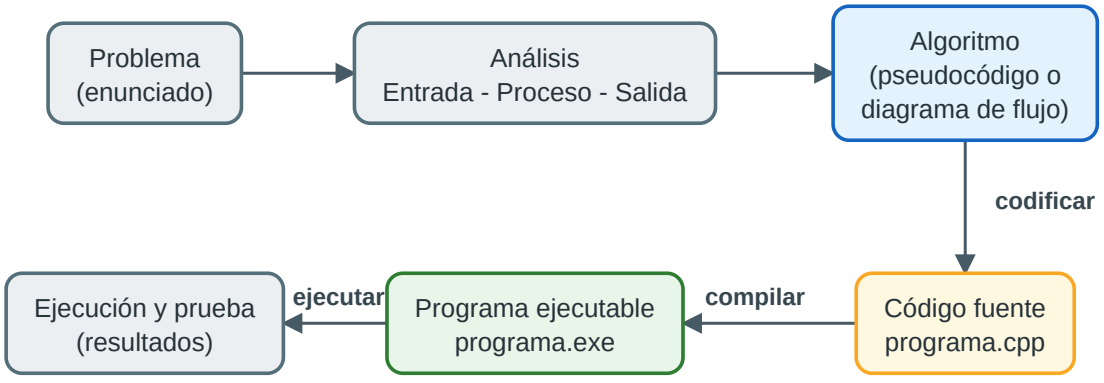
2.2 Código fuente

El **código fuente** es el algoritmo **escrito en un lenguaje de programación** (en nuestro caso, C++), respetando sus reglas. Es un archivo de texto que una persona puede leer y modificar; en C++ tiene la extensión **`.cpp`** (por ejemplo, `promedio.cpp`).

2.3 Programa ejecutable

El **programa ejecutable** es el resultado de **traducir (compilar)** el código fuente a lenguaje máquina (ceros y unos). Es el archivo que la computadora puede ejecutar directamente; en Windows tiene la extensión **`.exe`** (por ejemplo, `promedio.exe`). Ya no se puede leer como texto.

	Algoritmo	Código fuente	Programa ejecutable
¿Qué es?	La idea: los pasos para resolver el problema	Esos pasos escritos en C++	El código traducido a lenguaje máquina
¿Quién lo entiende?	Cualquier persona	Programadores (y el compilador)	Solo la computadora
¿Cómo se ve?	Lista de pasos, pseudocódigo o diagrama de flujo	Texto: `cout << "Hola";`	Binario: `10110000 01100001 …`
Ejemplo de archivo	Una hoja de papel	`promedio.cpp`	`promedio.exe`
¿Depende del lenguaje?	No	Sí (C++)	Sí (y del sistema operativo)



Del problema al programa: cada etapa produce algo distinto

> **Importante**
>
> **Respuesta modelo para el examen:** el *algoritmo* es la secuencia de pasos para resolver el problema (independiente del lenguaje); el *código fuente* es ese algoritmo escrito en un lenguaje de programación, como C++ (archivo `.cpp`); y el *programa ejecutable* es el código fuente traducido por el compilador a lenguaje máquina (archivo `.exe`), listo para que la computadora lo ejecute.

3. Lenguajes de programación

3.1 ¿Qué es un lenguaje de programación?

Es un **lenguaje formal**, con reglas estrictas, que sirve para escribir instrucciones que una computadora pueda ejecutar. Como todo lenguaje, tiene dos aspectos:

- **Sintaxis**: las reglas de *escritura* (cómo se escribe). Ejemplo: en C++ cada instrucción termina en punto y coma `;` y los textos van entre comillas dobles.
- **Semántica**: el *significado* (qué hace cada instrucción). Ejemplo: `x = 5;` significa «guardar el valor 5 en la variable `x`».

Una frase puede tener buena sintaxis y aun así ser incorrecta en su significado: `promedio = n1 + n2 + n3 / 3;` está bien escrita, pero calcula otra cosa (ver *precedencia* en la Parte II).

3.2 Niveles de los lenguajes

Nivel	Ejemplo	Ventajas	Desventajas
Lenguaje máquina	`10110000 01100001`	La CPU lo entiende directamente	Prácticamente ilegible para personas
Bajo nivel (ensamblador)	`MOV AL, 61h`	Control total del hardware, muy rápido	Difícil; cambia según el procesador
Alto nivel	C++, Java, Python	Fácil de leer y escribir; funciona en distintas computadoras	Necesita ser traducido

C++ es un lenguaje de alto nivel, pero también permite trabajar muy cerca del hardware (memoria, bits); por eso a veces se le llama «de nivel medio». Es muy usado en ingeniería electrónica (microcontroladores, sistemas embebidos), videojuegos, sistemas operativos y navegadores.

3.3 Traductores: compilador e intérprete

Como la computadora solo entiende lenguaje máquina, el código de alto nivel debe ser **traducido**. Hay dos formas:

	Compilador	Intérprete
Cómo traduce	Todo el programa de una vez, **antes** de ejecutarlo	Línea por línea, **mientras** lo ejecuta
Resultado	Un archivo ejecutable (`.exe`)	No genera ejecutable
Velocidad	Muy rápido al ejecutar	Más lento
Errores de escritura	Se detectan todos al compilar	Aparecen cuando se llega a esa línea
Lenguajes	C, C++, Go, Rust	Python, JavaScript

C++ es un lenguaje compilado. (Java es un caso intermedio: se compila a *bytecode* y luego lo ejecuta una máquina virtual).

3.4 Paradigmas de programación

Un **paradigma** es un *estilo* de programar:

- **Programación estructurada (imperativa)**: el programa es una secuencia de instrucciones que se ejecutan en orden. Es lo que haremos en el parcial.
- **Programación orientada a objetos (POO)**: el programa se organiza en **clases** y **objetos** que agrupan datos y acciones. Es la base del diagrama UML del ejercicio 9.

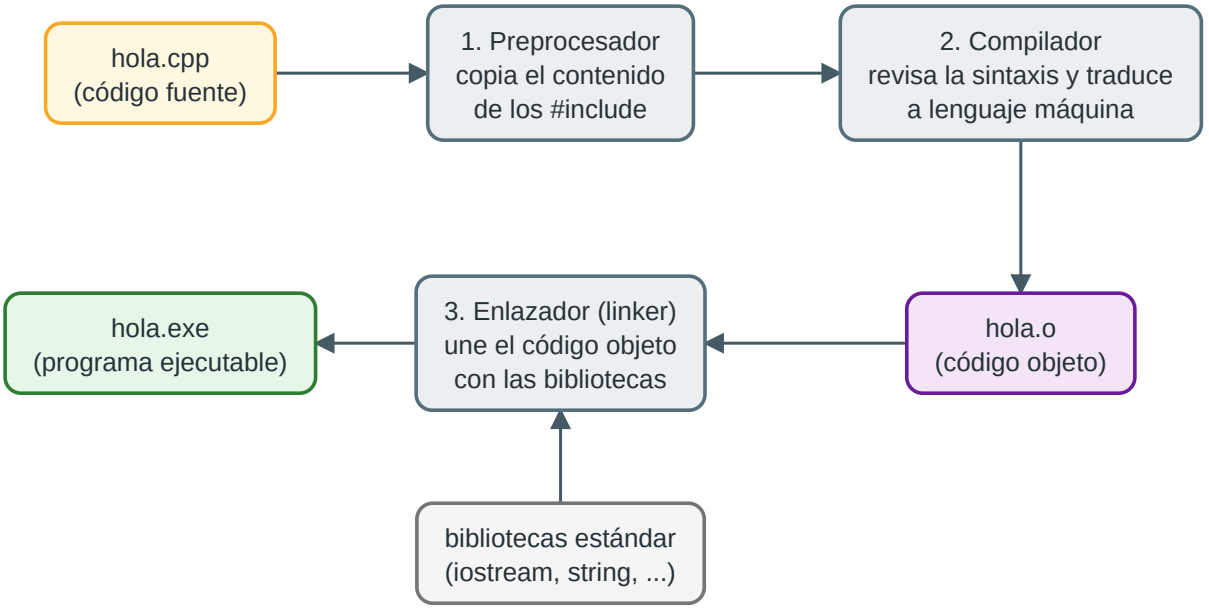
C++ es **multiparadigma**: permite ambos estilos.

3.5 Un poco de historia de C++

- **1972**: Dennis Ritchie crea el lenguaje **C** en los Laboratorios Bell.
- **1979–1985**: Bjarne Stroustrup crea **C++** como «C con clases» (C más programación orientada a objetos). El nombre viene del operador `++`, que significa «sumar uno»: *C++ es «un paso más» que C*.
- Hay versiones estándar que lo actualizan: C++98, C++11, C++14, C++17, C++20, C++23.

3.6 ¿Cómo se convierte un `.cpp` en un `.exe`? (compilación)

Cuando presionas «Compilar» en tu entorno (Dev-C++, Code::Blocks, etc.), ocurren tres etapas:



Las tres etapas de la compilación en C++

1. **Preprocesador**: lee las líneas que empiezan con `#` (como `#include <iostream>`) y copia en tu programa el contenido de esas bibliotecas.
2. **Compilador**: revisa que no haya errores de sintaxis y traduce el código a lenguaje máquina. Si encuentra un error (por ejemplo, falta un `;`), **se detiene y no genera el ejecutable**.
3. **Enlazador** (*linker*): une tu código traducido con el código de las bibliotecas (por ejemplo, el que hace funcionar a `cout`) y produce el **programa ejecutable**.

A todo el proceso, en el día a día, se le llama simplemente «compilar».

3.7 Tipos de errores

Tipo de error	Cuándo aparece	Ejemplo	Quién lo detecta
De sintaxis (de compilación)	Al compilar	Olvidar un `;`, escribir `Cout` en lugar de `cout`, no cerrar una comilla	El compilador: muestra el error y no crea el `.exe`
De ejecución	Mientras el programa corre	Pedir `substr(50, 3)` a un texto de 10 letras; dividir un entero entre 0	El programa se detiene («se cae»)
Lógico	Nunca «avisa»	Usar `a / b` con enteros esperando decimales; escribir mal una fórmula	Solo tú, probando el programa (prueba de escritorio)

> **Cuidado**
>
> Los errores **lógicos** son los más peligrosos: el programa compila, corre y muestra un resultado… **equivocado**. Por eso siempre hay que probar con datos cuyo resultado ya conoces.

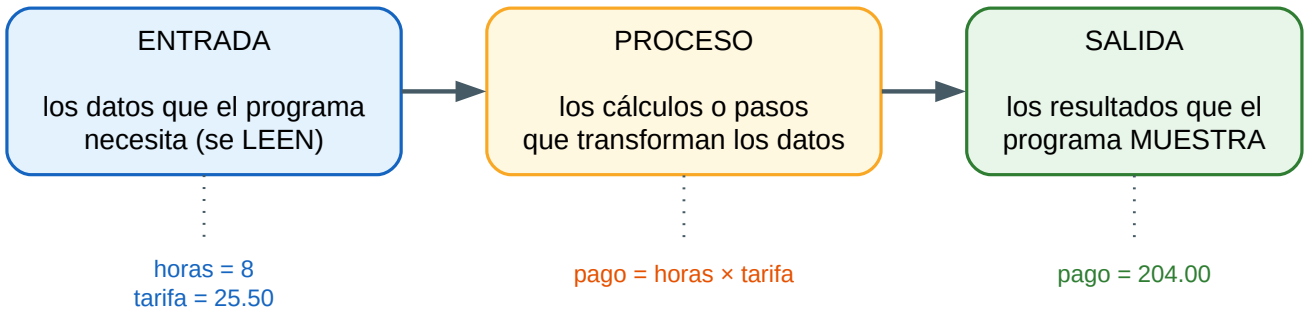
4. Metodología para resolver un problema

Antes de escribir código, un buen programador sigue estos pasos:

1. **Entender el problema**: leer el enunciado con calma. ¿Qué me dan? ¿Qué me piden?
2. **Analizar**: identificar entradas, proceso y salidas (E-P-S) y los tipos de datos.
3. **Diseñar el algoritmo**: escribir el pseudocódigo o dibujar el diagrama de flujo.
4. **Codificar**: traducir el algoritmo a C++.
5. **Probar**: hacer la prueba de escritorio y ejecutar el programa con datos conocidos.
6. **Documentar**: agregar comentarios que expliquen el código.

4.1 Análisis Entrada – Proceso – Salida (E-P-S)

Casi todos los programas del curso tienen la misma forma: **reciben datos, los procesan y muestran resultados**.



Esquema Entrada – Proceso – Salida con el ejemplo del pago de un trabajador

Para identificar cada parte, hazte estas preguntas:

Parte	Pregunta clave	Pistas en el enunciado
Entrada	¿Qué datos necesita el programa que **no conoce** de antemano?	«se ingresa», «el usuario escribe», «el programa recibe», «se solicita»
Proceso	¿Qué **cálculos** o transformaciones hay que hacer?	Fórmulas, «calcular», «extraer», «reemplazar»
Salida	¿Qué hay que **mostrar**?	«mostrar», «imprimir», «el resultado esperado es»

Es buena práctica presentar el análisis en una tabla que incluya el **tipo de dato** de cada variable y **por qué** se eligió:

Variable	Rol	Tipo	Justificación
`horas`	Entrada	`int`	Las horas se cuentan en números enteros (8)
`tarifa`	Entrada	`double`	El pago por hora tiene decimales (25.50)
`pago`	Salida	`double`	Resulta de multiplicar por un número con decimales

5. Pseudocódigo

5.1 ¿Qué es?

El **pseudocódigo** es una forma de escribir un algoritmo con **frases cortas en español, estructuradas como un programa**, pero **sin las reglas estrictas** de un lenguaje real (no lleva `;`, ni `<<`, ni llaves). Sirve para pensar la solución antes de programar y es independiente del lenguaje: el mismo pseudocódigo se puede traducir a C++, Java o Python.

5.2 Palabras y símbolos que usaremos

Usaremos el estilo de **PSeInt**, muy común en las universidades peruanas:

Pseudocódigo	Significado	Equivalente en C++
`Algoritmo Nombre` … `FinAlgoritmo`	Inicio y fin del algoritmo	`int main() {` … `return 0; }`
`Definir edad Como Entero`	Declarar una variable entera	`int edad;`
`Definir nota Como Real`	Variable con decimales	`double nota;`
`Definir nombre Como Cadena`	Variable de texto	`string nombre;`
`Definir inicial Como Caracter`	Un solo carácter	`char inicial;`
`Definir aprobado Como Logico`	Verdadero / falso	`bool aprobado;`
`Leer edad`	Pedir un dato al usuario	`cin >> edad;`
`Leer nombre` (con espacios)	Pedir una línea de texto	`getline(cin, nombre);`
`Escribir "Total: ", total`	Mostrar en pantalla	`cout << "Total: " << total;`
`total ← precio * cantidad`	Asignación (guardar el resultado)	`total = precio * cantidad;`
`// comentario`	Nota para quien lee	`// comentario`
`Si cond Entonces … SiNo … FinSi`	Decisión (*no se usa en el parcial*)	`if (cond) { … } else { … }`
`Mientras cond Hacer … FinMientras`	Repetición (*no se usa en el parcial*)	`while (cond) { … }`

Para trabajar con textos usaremos estas funciones (escritas igual que en C++ para no confundirnos):

Pseudocódigo	Qué devuelve	C++
`longitud(texto)`	Cantidad de caracteres	`texto.length()`
`buscar(texto, "@")`	Posición donde aparece `"@"`	`texto.find("@")`
`subcadena(texto, inicio, cantidad)`	Un pedazo del texto	`texto.substr(inicio, cantidad)`
`reemplazar(texto, inicio, cantidad, nuevo)`	El texto con un pedazo cambiado	`texto.replace(inicio, cantidad, nuevo)`

> **Nota**
>
> Si practicas en PSeInt, ten cuidado: su función `Subcadena(texto, desde, hasta)` pide la posición **final**, no la **cantidad** de caracteres como `substr` de C++. En el examen, escribe el pseudocódigo con la idea de `substr` (inicio y cantidad) y aclara qué significa cada número.

5.3 Reglas de un buen pseudocódigo

1. Una instrucción por línea.
2. Declarar las variables con su tipo al inicio.
3. Usar nombres significativos (`precioUnitario` , no `p`).
4. Sangrar (dejar espacios a la izquierda) el contenido entre `Algoritmo` y `FinAlgoritmo` .
5. No mezclar detalles de C++ (`;` , `<<` , `cout`).

5.4 Ejemplo completo: área de un rectángulo

Pseudocódigo:

```
Algoritmo AreaRectangulo
    Definir base, altura, area Como Real
    Escribir "Ingrese la base: "
    Leer base
    Escribir "Ingrese la altura: "
    Leer altura
    area ← base * altura
    Escribir "El area es: ", area
FinAlgoritmo
```

El mismo algoritmo en C++:

```cpp
#include <iostream>
using namespace std;

int main() {
    double base, altura, area;
    cout << "Ingrese la base: ";
    cin >> base;
    cout << "Ingrese la altura: ";
    cin >> altura;
    area = base * altura;
    cout << "El area es: " << area << endl;
    return 0;
}
```

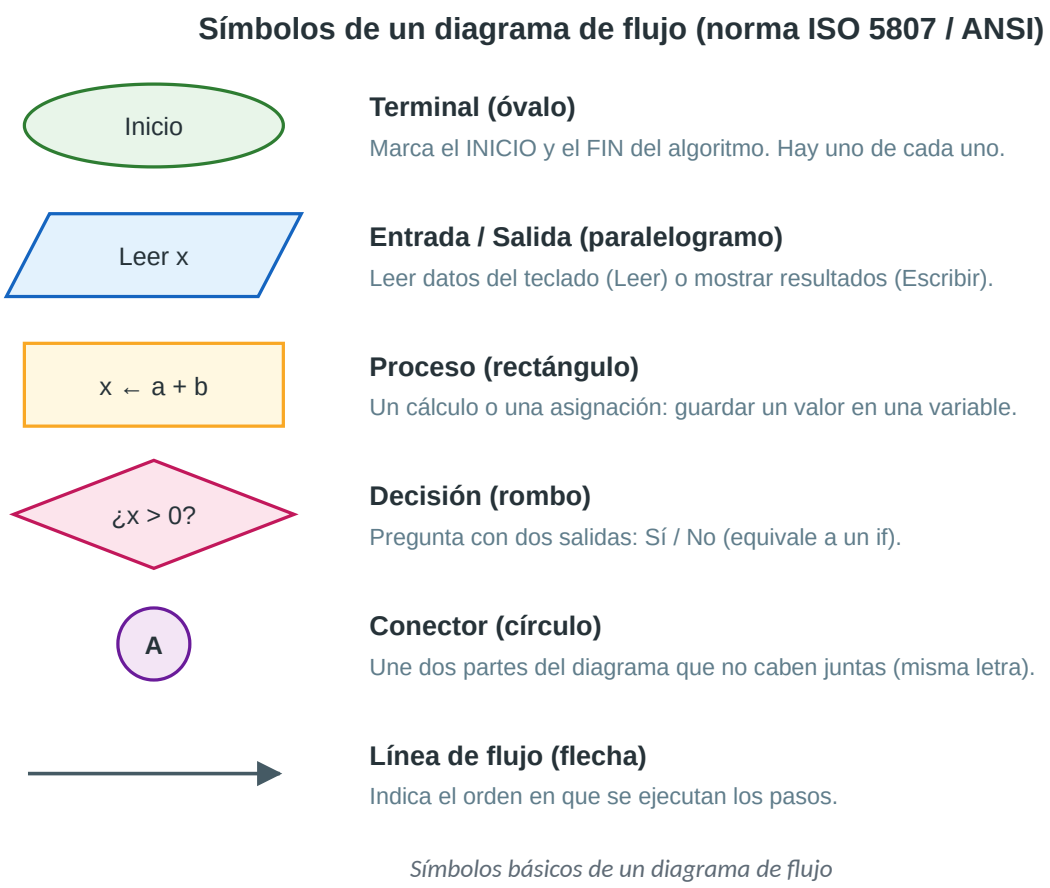
Fíjate en que **cada línea del pseudocódigo se convierte en una línea de C++**. Esa es la gran ventaja de hacer primero el pseudocódigo: después solo hay que «traducir».

6. Diagramas de flujo

6.1 ¿Qué es?

Un **diagrama de flujo** es la **representación gráfica de un algoritmo**: cada tipo de paso se dibuja con una figura distinta y las flechas indican el orden en que se ejecutan.

6.2 Símbolos



Símbolos básicos de un diagrama de flujo

> **Nota**
>
> Algunos profesores usan un símbolo distinto para la salida por pantalla (una figura con forma de monitor) o para la impresión (una hoja con borde ondulado). El paralelogramo para **leer y escribir** es la forma estándar y siempre es aceptada. Lo importante es escribir dentro **«Leer …»** o **«Escribir …»**.

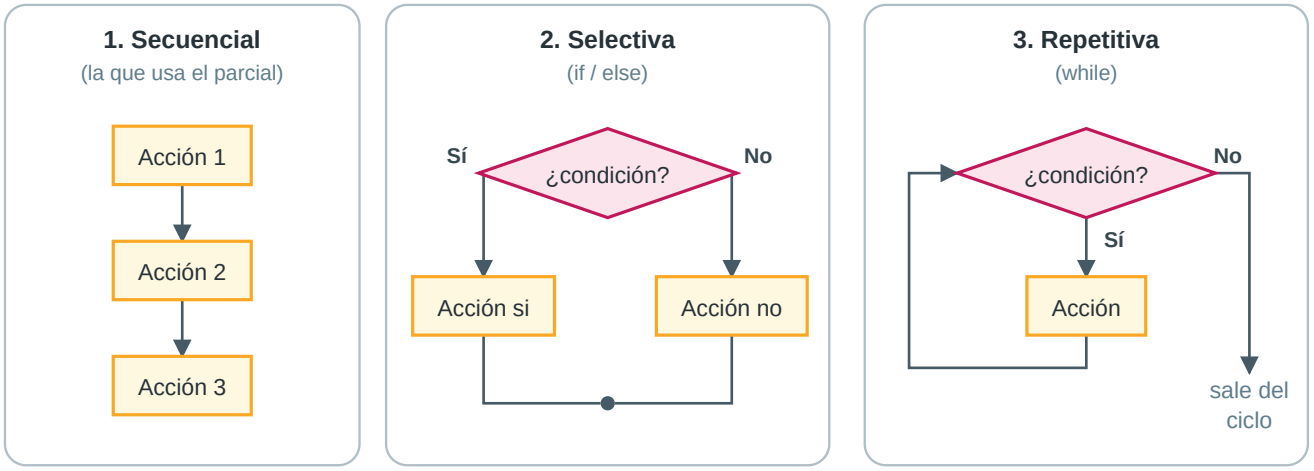
6.3 Reglas para dibujarlo bien

1. Tiene **un solo Inicio y un solo Fin**.
2. Se dibuja **de arriba hacia abajo** (y de izquierda a derecha).
3. Las líneas de flujo son **rectas** (horizontales o verticales) y terminan en **flecha**.
4. Las líneas **no deben cruzarse**; si es necesario, se usan conectores.
5. Cada símbolo tiene **una sola flecha de salida**, excepto la decisión, que tiene **dos** (Sí / No).
6. El texto dentro de cada símbolo es **breve y claro**.

7. Todo lo que se usa (variables) debe haberse leído o calculado antes.

6.4 Las tres estructuras básicas

Cualquier algoritmo, por complejo que sea, se arma combinando solo tres estructuras:



Estructuras secuencial, selectiva y repetitiva

1. **Secuencial**: los pasos se ejecutan uno tras otro, en orden. **Todos los ejercicios del parcial usan solo esta estructura** (la hoja dice «No es necesario utilizar `if`, `switch`, `for`, `while` ni `do-while`»).
2. **Selectiva**: se hace una pregunta y, según la respuesta, se toma un camino u otro (`if`).
3. **Repetitiva**: un grupo de pasos se repite mientras se cumpla una condición (`while`, `for`).

6.5 Ejemplo: área de un rectángulo

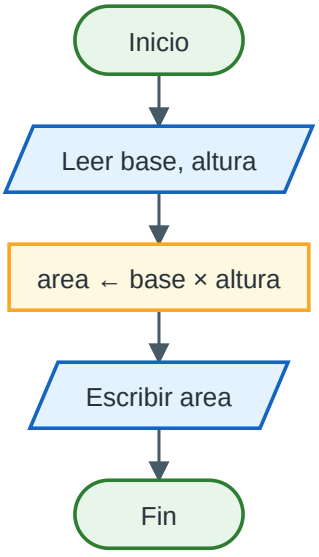


Diagrama de flujo del área de un rectángulo

6.6 Del diagrama al código

Símbolo	Lo que dice dentro	Se traduce a C++ como
Terminal	Inicio / Fin	`int main() {` … `return 0; }`
Entrada	`Leer base`	`cin >> base;`
Proceso	`area ← base × altura`	`area = base * altura;`
Salida	`Escribir area`	`cout << area;`

7. Prueba de escritorio

Una **prueba de escritorio** (también llamada *traza*) consiste en **ejecutar el algoritmo a mano**, con datos de prueba, anotando en una tabla **el valor de cada variable después de cada instrucción**. Sirve para encontrar errores lógicos *antes* de usar la computadora, y es exactamente lo que pide el ejercicio 10.

Cómo hacerla:

1. Elige datos de prueba cuyo resultado conozcas (o puedas calcular a mano).
2. Haz una tabla: una columna por variable y una para lo que se muestra en pantalla.
3. Recorre el algoritmo instrucción por instrucción y anota los cambios.
4. Las variables que todavía no tienen valor se marcan con `?` (en C++ contienen «basura»).

Ejemplo con el área del rectángulo, para `base = 5` y `altura = 3`:

Paso	Instrucción	`base`	`altura`	`area`	Pantalla
1	`Definir base, altura, area`	?	?	?	
2	`Leer base`	5	?	?	
3	`Leer altura`	5	3	?	
4	`area ← base * altura`	5	3	15	
5	`Escribir "El area es: ", area`	5	3	15	`El area es: 15`

Si el resultado de la tabla coincide con lo esperado (5 × 3 = 15), el algoritmo es correcto para ese caso.

8. UML básico: el diagrama de clases

8.1 ¿Qué es UML?

UML (*Unified Modeling Language*, Lenguaje Unificado de Modelado) es un **lenguaje gráfico estándar** para dibujar el diseño de un programa, así como un arquitecto dibuja los planos de una casa antes de construirla. UML tiene muchos tipos de diagramas; en el curso usamos el **diagrama de clases**.

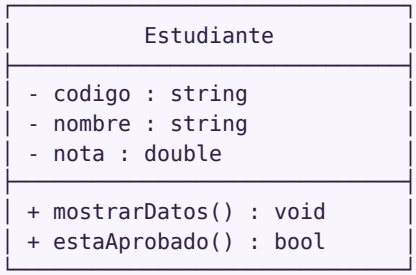
8.2 Clase y objeto

- Una **clase** es un **molde** o plantilla que describe cómo será un tipo de cosa: qué **datos** tiene y qué **acciones** puede hacer. Ejemplo: la clase `Estudiante`.
- Un **objeto** es un **ejemplar concreto** creado a partir de la clase, con sus propios valores. Ejemplo: el estudiante «Ana Torres, código 202601234, nota 15.75».

Analogía: la clase es el **plano** de una casa; los objetos son las **casas construidas** con ese plano. Todas tienen la misma forma, pero cada una tiene su propio color y sus propios dueños.

8.3 Partes de un diagrama de clases

Una clase se dibuja como un rectángulo dividido en **tres compartimentos**:



- **Atributo**: `visibilidad nombre : tipo` → `- nota : double`
- **Operación**: `visibilidad nombre(parámetros) : tipo_de_retorno` → `+ estaAprobado() : bool`
- **Visibilidad**:

Símbolo	Nombre	Significado	En C++
`+`	Público	Se puede usar desde fuera de la clase	`public:`
`-`	Privado	Solo la propia clase puede usarlo	`private:`
`#`	Protegido	La clase y sus «clases hijas»	`protected:`

- **Tipo de retorno**: lo que la operación devuelve. `void` significa «no devuelve nada» (por ejemplo, solo muestra datos en pantalla).

> **Consejo**
>
> Regla práctica que casi siempre se usa: **atributos privados (`-`)** y **operaciones públicas (`+`)**. Así nadie puede cambiar los datos «a escondidas»; solo a través de las operaciones de la clase.

8.4 Del diagrama UML al código C++

En el diagrama UML	En C++
Nombre de la clase `Producto`	`class Producto { … };`
`- precio : double`	dentro de `private:` → `double precio;`
`+ calcularValorStock() : double`	dentro de `public:` → `double calcularValorStock() { … }`
Un objeto `p1 : Producto`	`Producto p1(…);` en `main`

Todo esto se explica con código en la sección 16 (Parte II) y se aplica en el ejercicio 9.

Parte II — C++ desde cero

9. Tu primer programa en C++

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hola, mundo" << endl;
    return 0;
}
```

Al ejecutarlo, en la pantalla aparece:

```
Hola, mundo
```

Veamos qué significa cada línea:

Línea	Qué hace
`#include <iostream>`	Le dice al preprocesador que incluya la biblioteca **iostream** (*input/output stream*), que trae `cin` y `cout`. Las líneas con `#` **no llevan** `;`.
`using namespace std;`	Permite escribir `cout` en lugar de `std::cout`. Todo lo de la biblioteca estándar vive en el «espacio de nombres» `std`.
`int main() {`	Es la **función principal**: el programa **siempre empieza a ejecutarse aquí**. `int` indica que, al terminar, devuelve un número entero al sistema.
`cout << "Hola, mundo" << endl;`	Muestra el texto en pantalla. `<<` «envía» lo que está a su derecha hacia la pantalla. `endl` produce un salto de línea.
`return 0;`	Termina `main` y avisa al sistema que todo salió bien (0 = sin errores).
`}`	Cierra el bloque de `main`.

Reglas básicas de escritura

- **Cada instrucción termina en punto y coma** `;` (salvo las líneas `#include` y las que abren o cierran llaves).
- C++ **distingue mayúsculas de minúsculas**: `cout` ≠ `Cout` ≠ `COUT`. Lo mismo con las variables: `nota` y `Nota` son distintas.
- Los textos van entre **comillas dobles**: `"Hola"`.
- Las **llaves** `{ }` agrupan instrucciones en un bloque.
- **Comentarios**: el compilador los ignora; sirven para explicar el código.
  - De una línea: `// esto es un comentario`
  - De varias líneas: `/* esto también */`
- Los espacios y la sangría no le importan al compilador, pero **sí a quien lee**: ordena tu código.

Plantilla para todos los ejercicios del parcial

Todos los programas del parcial siguen el mismo esqueleto. Apréndetelo:

```cpp
#include <iostream>   // cin, cout
#include <string>     // string, getline (si se usan textos)
#include <iomanip>    // fixed, setprecision (si se piden decimales)
using namespace std;

int main() {
    // 1. Declarar las variables (con el tipo adecuado)

    // 2. ENTRADA: pedir los datos (cout + cin / getline)

    // 3. PROCESO: hacer los cálculos

    // 4. SALIDA: mostrar los resultados (cout)

    return 0;
}
```

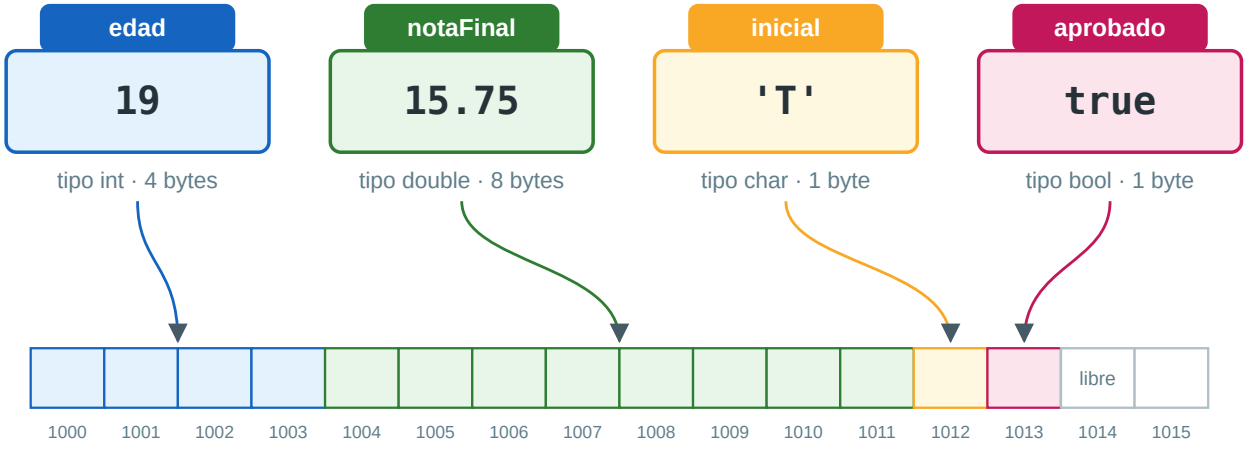
10. Variables y tipos de datos

10.1 ¿Qué es una variable?

Una **variable** es un **espacio en la memoria RAM** donde se guarda un dato que puede cambiar durante la ejecución. Imagínala como una **caja con etiqueta**:

- la **etiqueta** es el *nombre* de la variable (`edad`),
- el **tamaño y la forma** de la caja dependen del *tipo* (`int`),
- lo que hay **dentro** es el *valor* (`19`).

Una variable es una «caja» con nombre, tipo y valor guardada en la memoria RAM



Memoria RAM: cada casilla es 1 byte; el número de abajo es su dirección

Esquema simplificado: el compilador decide las direcciones exactas.

Cada variable ocupa uno o más bytes en la memoria según su tipo

10.2 Declarar, inicializar y asignar

```
int edad;          // DECLARAR: se crea la caja (todavía tiene "basura")
edad = 18;         // ASIGNAR: se guarda un valor
int nota = 15;     // DECLARAR + INICIALIZAR en una sola línea
edad = edad + 1;   // se lee el valor actual (18), se le suma 1 y se guarda: 19
```

- El signo `=` **no** es la igualdad de matemática: significa **«guarda en la variable de la izquierda el resultado de la derecha»**. Por eso `edad = edad + 1;` tiene sentido en programación.
- Una variable declarada pero **no inicializada** contiene un valor impredecible («basura»). Nunca la uses antes de darle un valor.
- Se pueden declarar varias variables del mismo tipo en una línea: `double subtotal, descuento, total;`

10.3 Reglas para los nombres (identificadores)

Regla	Válido	Inválido
Solo letras, dígitos y guion bajo `_`	`nota_final`, `nota1`	`nota-final`, `nota@`
No puede empezar con un dígito	`nota1`	`1nota`
Sin espacios	`notaFinal`	`nota final`
No usar palabras reservadas	`total`	`int`, `double`, `return`
Evitar tildes y la ñ	`anio`, `codigo`	`año`, `código`

Además:

- **Mayúsculas importan**: `Precio` y `precio` son variables distintas.
- Usa **nombres significativos**: `precioUnitario` en vez de `p` (la hoja lo pide: «Utilice nombres de variables significativos»).
- Estilo recomendado: **camelCase** (`precioUnitario`, `notaFinal`). Las constantes suelen ir en MAYÚSCULAS (`IGV`).

10.4 Tipos de datos fundamentales

Tipo	Qué guarda	Ejemplos de valores	Tamaño típico	Rango aproximado
`int`	Números **enteros**	`19`, `-5`, `2026`	4 bytes	−2 147 483 648 a 2 147 483 647
`double`	Números **reales** (con decimales), ~15 cifras de precisión	`15.75`, `25.50`, `-0.5`	8 bytes	$\pm 1.7 \times 10^{308}$
`float`	Reales con **menos** precisión (~7 cifras)	`3.14f`	4 bytes	$\pm 3.4 \times 10^{38}$
`char`	**Un solo carácter**	`'A'`, `'7'`, `'@'`	1 byte	256 valores
`bool`	**Verdadero o falso**	`true`, `false`	1 byte	2 valores
`string`	**Texto** (cadena de caracteres)	`"Juan Perez"`	Variable	Lo que quepa en memoria

`string` no es un tipo «primitivo» del lenguaje: viene de la biblioteca estándar y necesita `#include <string>`. Otros tipos que existen: `long long` (enteros muy grandes), `short` (enteros pequeños), `unsigned int` (enteros sin signo).

10.5 ¿Cómo elegir el tipo correcto?

Hazte estas preguntas, en este orden:

1. ¿Es **texto** (nombres, direcciones, palabras)? → `string`. ¿Es **un único carácter**? → `char`.
2. ¿Es un número con el que **no se hacen cálculos** (DNI, código, teléfono, código de matrícula)? → `string`. Puede empezar con 0, puede tener letras o guiones, y no tiene sentido sumarlo.
3. ¿Es una **cantidad entera** (edad, unidades, años de antigüedad)? → `int`.
4. ¿Puede tener **decimales** (precio, nota, sueldo, porcentaje, medida)? → `double`.
5. ¿Es **sí/no**, **verdadero/falso**? → `bool`.

> **Cuidado**
>
> En C++, un número entero que empieza con `0` se interpreta en **base 8 (octal)**: `int x = 0145;` guarda **101**, ¡no 145! Es otra razón para guardar códigos como `"0145"` en un `string`.

10.6 Literales: cómo se escriben los valores

Literal	Tipo	Comentario
`25`	`int`	Sin punto decimal
`25.0`, `25.50`, `2.5e3`	`double`	Con punto decimal (`2.5e3` = $2.5 \times 10^3 = 2500$)
`25.0f`	`float`	Con `f` al final
`'A'`	`char`	**Comillas simples**, un solo carácter
`"A"`, `"Hola"`	texto	**Comillas dobles**
`true`, `false`	`bool`	En minúsculas

> **Trampa de examen**
>
> `char letra = "A";` es un **error** (comillas dobles para un `char`), y `'Hola'` también (comillas simples para varias letras). Recuerda: **simples → un carácter; dobles → texto**. En los decimales se usa **punto**, nunca coma: `25.50`, no `25,50`.

10.7 Constantes

Si un valor **no debe cambiar nunca**, se declara como constante con `const`:

```
const double IGV = 0.18;    // impuesto del 18 %
const int MESES = 12;
// IGV = 0.20;   // ERROR de compilación: una constante no se puede modificar
```

10.8 ¿Cuánta memoria ocupa cada tipo?

El operador `sizeof` devuelve cuántos bytes ocupa un tipo. Esto es lo que muestra el programa del ejercicio 1:

```
--- Bytes que ocupa cada tipo en memoria ---
int    : 4 bytes
double : 8 bytes
char   : 1 byte
bool   : 1 byte
```

11. Entrada y salida de datos

11.1 Mostrar datos: `cout <<`

`cout` (*console output*) envía datos a la pantalla. Se pueden encadenar varios `<<`:

```
int edad = 19;
cout << "Tengo " << edad << " anios" << endl;   // Tengo 19 anios
```

Secuencias de escape útiles dentro de un texto:

Secuencia	Significado
`\n`	Salto de línea (igual que `endl`)
`\t`	Tabulación
`\"`	Comillas dobles dentro de un texto
`\\`	Una barra invertida

`endl` y `"\n"` hacen casi lo mismo (saltar de línea); usa el que prefieras.

11.2 Leer datos: `cin >>`

`cin` (*console input*) lee lo que el usuario escribe y lo guarda en una variable, convirtiéndolo al tipo de esa variable:

```
int horas;
double tarifa;
cout << "Horas trabajadas: ";
cin >> horas;              // el usuario escribe 8 y presiona Enter
cout << "Pago por hora: ";
cin >> tarifa;             // el usuario escribe 25.50
```

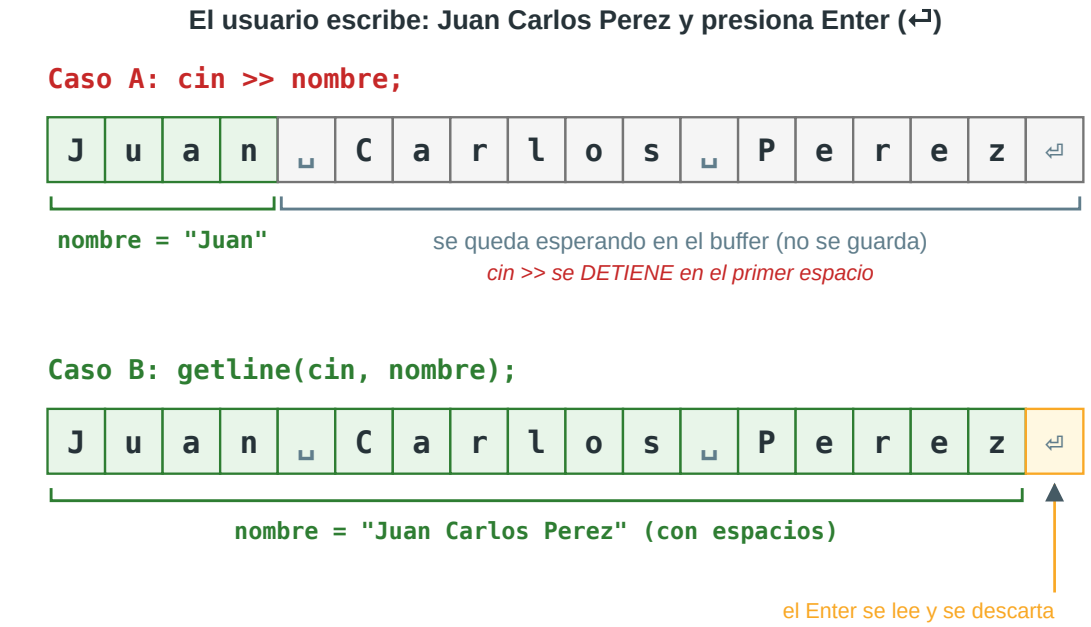
Se pueden leer varios datos en una sola instrucción: `cin >> horas >> tarifa;` (el usuario los separa con espacios o Enter).

> **Importante**
>
> `cin >>` **lee hasta el primer espacio en blanco** (espacio, tabulación o Enter). Sirve para números y para palabras sueltas, pero **no para textos con espacios** como un nombre completo.

11.3 Leer textos con espacios: `getline`

`getline(cin, variable);` lee **toda la línea**, incluidos los espacios, hasta que el usuario presiona Enter. Necesita `#include <string>`.

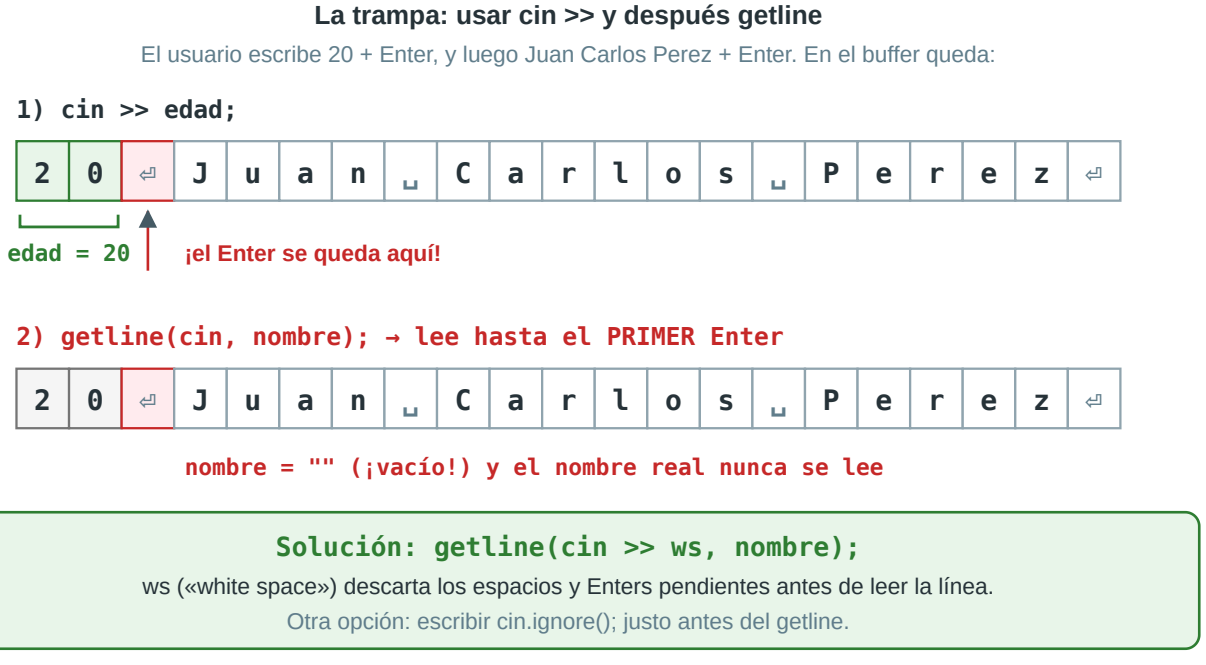
```
string nombre;
cout << "Ingrese nombre completo: ";
getline(cin, nombre);      // "Juan Carlos Perez" completo
```



Qué lee cin >> y qué lee getline cuando el usuario escribe un nombre con espacios

11.4 La trampa de mezclar `cin >>` y `getline`

Cuando presionas Enter, ese Enter también es un carácter (`'\n'`) y queda guardado en una «sala de espera» llamada **buffer de entrada**. `cin >>` lee el número pero **deja el Enter en el buffer**. Si después viene un `getline`, este encuentra el Enter pendiente, cree que la línea ya terminó y **lee un texto vacío**.



Solución: `getline(cin >> ws, nombre);`

ws («white space») descarta los espacios y Enters pendientes antes de leer la línea.

Otra opción: escribir cin.ignore(); justo antes del getline.

La trampa: después de cin >> queda un Enter pendiente que getline toma como línea vacía

Hay tres soluciones:

Solución	Código	Cuándo usarla
1. `ws` (recomendada)	`getline(cin >> ws, nombre);`	Siempre que un `getline` venga después de un `cin >>`
2. `cin.ignore()`	`cin.ignore();` y luego `getline(cin, nombre);`	Ignora un carácter (el Enter)
3. Cambiar el orden	Leer primero todos los textos con `getline` y al final los números con `cin >>`	Cuando el enunciado lo permite (lo hacemos en los ejercicios 8 y 10)

Resumen: ¿cuál uso?

Dato a leer	Instrucción
Número (`int` , `double`)	`cin >> variable;`
Una sola palabra o código sin espacios	`cin >> variable;`
Texto que puede tener espacios (nombre completo, producto, dirección)	`getline(cin, variable);`

12. Operadores y expresiones

Una **expresión** es una combinación de valores, variables y operadores que produce un resultado: `precio * cantidad`, `a + b * c`.

12.1 Operadores aritméticos

Operador	Operación	Ejemplo con enteros	Ejemplo con reales
`+`	Suma	`7 + 2` → `9`	`7.5 + 2` → `9.5`
`-`	Resta	`7 - 2` → `5`	`7.5 - 2` → `5.5`
`*`	Multiplicación	`7 * 2` → `14`	`7.5 * 2` → `15`
`/`	División	`7 / 2` → **`3`**	`7.0 / 2` → `3.5`
`%`	Módulo (resto)	`7 % 2` → `1`	*(no se usa con reales)*

12.2 La división entera (¡muy importante!)

Cuando **los dos operandos son enteros**, `/` hace **división entera**: calcula el cociente y **descarta los decimales** (no redondea, *corta*).

Expresión	Resultado	¿Por qué?
`7 / 2`	`3`	3.5 → se corta el .5
`10 / 4`	`2`	2.5 → se corta el .5
`1 / 2`	`0`	0.5 → se corta todo
`-7 / 2`	`-3`	Se corta hacia el cero
`7.0 / 2`	`3.5`	Hay un `double` : división real

Trampa de examen

Una fórmula como `porcentaje / 100` **da 0** si `porcentaje` es `int` y vale menos de 100 (por ejemplo, `10 / 100` → `0`). Usa `porcentaje / 100.0` o declara `porcentaje` como `double`.

12.3 El operador módulo `%`

`a % b` devuelve el **resto** de la división entera de `a` entre `b`. Solo funciona con enteros.

Expresión	Resultado	Lectura
`8 % 3`	`2`	8 = 3 × 2 + **2**
`7 % 2`	`1`	7 es impar
`10 % 2`	`0`	10 es par
`-7 % 3`	`-1`	El signo sigue al dividendo
`2026 % 10`	`6`	El último dígito de un número

12.4 Asignación y operadores compuestos

Escrito así	Equivale a	Si `k` valía 10, queda
`k = 5;`	—	`5`
`k += 5;`	`k = k + 5;`	`15`
`k -= 3;`	`k = k - 3;`	`7`
`k *= 2;`	`k = k * 2;`	`20`
`k /= 4;`	`k = k / 4;`	`2`
`k++;`	`k = k + 1;`	`11`
`k--;`	`k = k - 1;`	`9`

12.5 Operadores relacionales y lógicos (solo para reconocerlos)

Comparan valores y devuelven `true` o `false`. En el parcial casi no se usan (no hay `if`), pero aparecen en todos los programas con decisiones:

- Relacionales: `==` (igual), `!=` (distinto), `<`, `>`, `<=`, `>=`.
- Lógicos: `&&` (y), `||` (o), `!` (no).

Ojo: `=` **asigna**; `==` **compara**.

12.6 Precedencia y asociatividad

Cuando una expresión tiene varios operadores, C++ no la evalúa de izquierda a derecha «porque sí»: sigue una **jerarquía** llamada **precedencia**, igual que en matemática («primero multiplicaciones, luego sumas»).

Prioridad	Operadores	Asociatividad (si hay empate)
1 (la más alta)	`( )` paréntesis	—
2	Unarios: `-x`, `!`, `++`, `--`, `static_cast<…>()`	Derecha a izquierda
3	`*` `/` `%`	**Izquierda a derecha**
4	`+` `-`	**Izquierda a derecha**
5	`<<` `>>` (también los de `cout` y `cin`)	Izquierda a derecha
6	`<` `<=` `>` `>=`	Izquierda a derecha
7	`==` `!=`	Izquierda a derecha
8	`&&`	Izquierda a derecha
9	`||`	Izquierda a derecha
10 (la más baja)	`=` `+=` `-=` `*=` `/=` `%=`	Derecha a izquierda

- **Precedencia**: decide *qué operador va primero* cuando son distintos (`*` antes que `+`).
- **Asociatividad**: decide *el orden cuando tienen la misma precedencia* (`/` y `*` → de izquierda a derecha).

Método para evaluar paso a paso — ejemplo: `2 + 3 * 4 - 6 / 2`

1. Primero `*` y `/`, de izquierda a derecha: `3 * 4 = 12` y `6 / 2 = 3` → queda `2 + 12 - 3`.
2. Luego `+` y `-`, de izquierda a derecha: `2 + 12 = 14`, `14 - 3 = 11`.
3. Resultado: **11**.

> **Consejo**
>
> Cuando tengas dudas, **usa paréntesis**: no cambian el resultado si ya respetan la precedencia, y hacen el código más fácil de leer. `a + (b * c)` se entiende mejor que `a + b * c`.

Un detalle con `cout`: `cout << a + b;` funciona porque `+` tiene mayor precedencia que `<<` (primero se suma y luego se muestra). En cambio, `cout << a < b;` da error de compilación: hay que escribir `cout << (a < b);`.

13. Conversión de tipos

13.1 Conversión implícita (automática)

C++ convierte tipos automáticamente en dos situaciones:

a) En una operación con tipos mezclados, el tipo «menor» se convierte al «mayor»: `int` con `double` → todo se hace en `double`.

```
int horas = 8;
double tarifa = 25.50;
double pago = horas * tarifa;    // 8 se convierte en 8.0 → 204.0
```

b) Al asignar, el valor se convierte al tipo de la variable que lo recibe:

```
int n = 9.99;      // n vale 9   (se CORTAN los decimales, no se redondea)
double m = 5;      // m vale 5.0
```

> **Importante**
>
> **La regla de oro:** C++ calcula primero el lado derecho **con los tipos que tiene ese lado** y recién después convierte el resultado al tipo de la variable. El tipo de la variable de la izquierda **no influye** en cómo se calcula:
>
> `double x = 10 / 4;` → primero `10 / 4` entre enteros = `2`, luego se guarda `2.0`. ¡Los decimales ya se perdieron!

13.2 Conversión explícita: `static_cast`

Cuando queremos convertir **a propósito**, usamos `static_cast<tipo>(valor)`:

```
int a = 7, b = 2;
double r1 = static_cast<double>(a) / b;   // 7.0 / 2 = 3.5   (correcto)
double r2 = (double)a / b;                // 3.5  (forma antigua, estilo C)
double r3 = static_cast<double>(a / b);   // 3.0  (¡tarde! primero 7/2 = 3)
double r4 = a / 2.0;                      // 3.5  (basta un literal double)
```

- `static_cast<double>(a)` **no cambia la variable** `a` (sigue siendo `int` y sigue valiendo 7): produce una **copia temporal** de su valor convertida a `double`, solo para esa expresión.
- Se prefiere `static_cast` a `(double)a` porque es más visible, más fácil de buscar en el código y el compilador revisa que la conversión tenga sentido.
- La posición del `static_cast` importa: hay que convertir **antes** de dividir (`r1`), no después (`r3`).

Resultados reales de esos ejemplos:

```
static_cast<double>(a) / b = 3.5
(double)a / b              = 3.5
static_cast<double>(a / b) = 3
a / 2.0                    = 3.5
```

14. Formato de salida con `<iomanip>`

Por defecto, `cout` muestra los `double` con **hasta 6 cifras significativas** y **sin ceros innecesarios**. Para controlar los decimales se usa la biblioteca `<iomanip>`:

Manipulador	Efecto	¿Dura?
`fixed`	Notación normal (no científica), con número fijo de decimales	Sí, hasta que se cambie
`setprecision(n)`	Junto con `fixed`: muestra **exactamente n decimales**	Sí
`setw(n)`	Reserva `n` espacios para **el siguiente** dato	**No**: solo afecta al siguiente
`left` / `right`	Alinea a la izquierda / derecha dentro de `setw`	Sí

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double a = 2.0, b = 204.0, c = 3.14159265, d = 1573.425, e = 1234567.0;

    cout << "Sin formato : " << a << "  " << b << "  " << c << "  " << d << "  " << e << endl;

    cout << fixed << setprecision(2);
    cout << "fixed + 2   : " << a << "  " << b << "  " << c << "  " << d << "  " << e << endl;

    cout << "[" << setw(8) << 3.5 << "]" << endl;
    cout << "[" << left << setw(8) << 3.5 << "]" << endl;
    return 0;
}
```

Salida real:

```
Sin formato : 2  204  3.14159  1573.42  1.23457e+06
fixed + 2   : 2.00  204.00  3.14  1573.42  1234567.00
[    3.50]
[3.50    ]
```

Observa que:

- Sin formato, `2.0` se muestra como `2` y `1234567.0` en notación científica (`1.23457e+06` = 1.23457×10^6).
- Con `fixed << setprecision(2)` **todos** los números siguientes salen con 2 decimales; se escribe **una sola vez**.
- `1573.425` se muestra como `1573.42`, no `1573.43`. No es un error de C++: la computadora no puede guardar exactamente 1573.425 en binario y guarda 1573.4249999999999..., que redondeado a 2 decimales da `.42`. Lo veremos en el ejercicio 8.

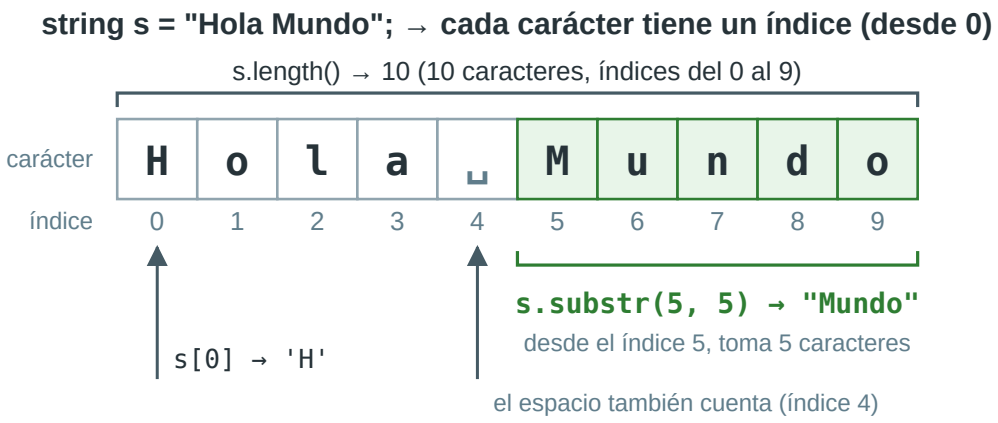
15. Cadenas de caracteres: `string`

15.1 Crear, leer y mostrar textos

```cpp
#include <string>
...
string carrera = "Ingenieria Electronica";   // inicializar
string nombre;                               // vacío
getline(cin, nombre);                        // leer con espacios
cout << nombre << " estudia " << carrera << endl;
```

15.2 Índices: la posición de cada carácter

Cada carácter de un `string` ocupa una posición numerada llamada **índice**, que **empieza en 0**. El último índice es siempre `longitud − 1`. Los espacios y los símbolos también cuentan.



Índices del texto "Hola Mundo"

15.3 Funciones principales

Función	Qué hace	Ejemplo con `s = "Hola Mundo"`	Resultado
`s.length()` o `s.size()`	Cantidad de caracteres	`s.length()`	`10`
`s[i]` o `s.at(i)`	El carácter de la posición `i`	`s[0]`, `s.at(5)`	`'H'`, `'M'`
`s.find(texto)`	Posición de la **primera** aparición	`s.find("Mundo")`	`5`
`s.find(texto, desde)`	Busca a partir de la posición `desde`	`s.find('o', 2)`	`9`
`s.substr(inicio, cantidad)`	Copia `cantidad` caracteres desde `inicio`	`s.substr(5, 5)`	`"Mundo"`
`s.substr(inicio)`	Copia desde `inicio` **hasta el final**	`s.substr(5)`	`"Mundo"`
`s.replace(inicio, cantidad, nuevo)`	**Modifica** `s`: borra `cantidad` caracteres desde `inicio` y pone `nuevo`	`s.replace(0, 4, "Chau")`	`s` pasa a ser `"Chau Mundo"`
`s + t`	Une (concatena) textos	`s + "!"`	`"Hola Mundo!"`
`s.insert(pos, texto)`	Inserta texto en una posición	`s.insert(4, ",")`	`"Hola, Mundo"`
`s.erase(inicio, cantidad)`	Borra caracteres	`s.erase(4, 6)`	`"Hola"`
`s.empty()`	¿Está vacío?	`s.empty()`	`false` (se muestra `0`)
`to_string(n)`	Número → texto	`to_string(2026)`	`"2026"`
`stoi(t)`, `stod(t)`	Texto → `int` / `double`	`stoi("0145") + 1`	`146`

Detalles que suelen preguntarse:

- **Si `find` no encuentra el texto**, devuelve la constante `string::npos` (un número enorme: en una computadora de 64 bits se muestra como `18446744073709551615`). Para saber si encontró algo se compara:

`s.find("xyz") == string::npos`.

- `find` devuelve un valor de tipo `size_t` (entero sin signo). En el curso es común guardarlo en un `int` (`int pos = s.find("@");`) y funciona bien con textos normales.
- Si en `substr` la **cantidad** se pasa del final, simplemente copia hasta el final: `s.substr(7, 50)` → `"ndo"`.
- Si el **inicio** está fuera del texto, el programa **se cae** (error de ejecución `out_of_range`): `"Hola".substr(10, 2)` termina con el mensaje `basic_string::substr: __pos (which is 10) > this->size() (which is 4)`.
- `replace`, `insert` y `erase` **modifican la variable original**. Si necesitas conservar el original, trabaja sobre una copia: `string copia = s;`.

15.4 Fórmulas útiles con `find` y `substr`

Si `p` es la posición de un separador (por ejemplo, la `@` de un correo o un guion):

Quiero...	Fórmula
Todo lo que está **antes** del separador	`s.substr(0, p)` (desde 0, tantos caracteres como `p`)
Todo lo que está **después** del separador	`s.substr(p + 1)`
Lo que hay **entre** dos posiciones `i` y `j` (sin incluirlas)	`s.substr(i + 1, j - i - 1)`
Los **últimos** `n` caracteres	`s.substr(s.length() - n)`
Cuántas letras tiene una palabra que voy a reemplazar	`string("universidad").length()` → `11`

> **Consejo**
>
> Antes de escribir un `substr`, **dibuja el texto con sus índices debajo** (como en las figuras de esta guía). En el examen, eso te evita casi todos los errores de «me salió una letra de más o de menos».

16. Agrupar datos: `struct` y `class`

16.1 `struct`: un «paquete» de datos

Un `struct` agrupa varias variables relacionadas bajo un solo nombre. Así, en lugar de tener `codigo1`, `nombre1`, `precio1`... tenemos un único **Producto** con todos sus datos.

```
struct Producto {
    string codigo;
    string nombre;
    double precio;
    int    stock;
};   // ¡ojo con el punto y coma al final!

int main() {
    Producto p;                 // p es una variable (objeto) de tipo Producto
    p.codigo = "PRD-0012";      // con el punto (.) se accede a cada dato
    p.precio = 45.90;
    cout << p.codigo << endl;
}
```

16.2 `class` : datos privados + operaciones públicas

Una `class` es como un `struct`, pero pensada para la programación orientada a objetos: normalmente sus **atributos son privados** y se trabaja con ellos mediante **métodos** (funciones que pertenecen a la clase).

```
class Producto {
private:                        // "-" en UML
    double precio;
    int    stock;
public:                         // "+" en UML
    Producto(double p, int s) { // constructor: da los valores iniciales
        precio = p;
        stock = s;
    }
    double calcularValorStock() {   // método (operación)
        return precio * stock;      // return: el valor que devuelve
    }
};
```

- `private:` → solo los métodos de la clase pueden usar esos datos.
- `public:` → se pueden usar desde `main`.
- El **constructor** es un método especial con el mismo nombre de la clase; se ejecuta automáticamente al crear el objeto.
- `return` indica el valor que el método devuelve; `void` significa que no devuelve nada.

16.3 Crear objetos y usar sus métodos

```
Producto p1(45.90, 30);                     // se crea el objeto (llama al constructor)
cout << p1.calcularValorStock() << endl;    // 1377
```

16.4 Diferencia entre `struct` y `class`

	`struct`	`class`
Visibilidad por defecto	Todo **público**	Todo **privado**
Uso típico	Agrupar datos simples	Objetos con datos protegidos y métodos
¿Puede tener métodos?	Sí	Sí

En la práctica, la **única diferencia técnica** es la visibilidad por defecto. Para el ejercicio 9, la versión «simplificada» que pide el enunciado es un `struct`; la traducción fiel del diagrama UML (con `-` y `+`) es una `class`. Ambas se resuelven en la Parte III.

Parte III — Los 10 ejercicios resueltos paso a paso

> **Consejo**
>
> Antes de leer cada solución, **intenta resolver el ejercicio tú solo, en papel**. Después compara. Equivocarse primero y luego entender por qué es la forma más rápida de aprender.

Ejercicio 1 — Tipos de datos y variables

Nivel: Básico · **Objetivo:** reforzar `string`, `int`, `double`, `bool` y `char`.

Enunciado. Para cada dato, indicar el tipo de dato C++ más apropiado y luego declarar las variables: nombre completo de un estudiante; edad; nota final con decimales; código de matrícula como `"202601234"`; estado de aprobación (verdadero/falso); inicial del apellido.

Qué se evalúa

Que sepas **elegir el tipo según la naturaleza del dato** (texto, entero, real, lógico, un carácter) y **justificar** la elección. Repasa las secciones 10.4 y 10.5 de la Parte II.

Solución: tipo de cada dato

Dato	Tipo C++	Justificación	Ejemplo
Nombre completo	`string`	Es texto y tiene espacios (se lee con `getline`)	`"Ana Lucia Torres Vega"`
Edad	`int`	Es un número entero: no tiene decimales	`19`
Nota final con decimales	`double`	Es un número real; `double` es el tipo estándar para decimales en C++ y tiene más precisión que `float`	`15.75`
Código de matrícula `"202601234"`	`string`	Aunque solo tiene dígitos, es un **identificador**: no se suma ni se promedia, puede tener ceros a la izquierda y se procesa por partes con `substr`	`"202601234"`
Estado de aprobación	`bool`	Solo tiene dos valores posibles: verdadero o falso	`true`
Inicial del apellido	`char`	Es un único carácter (va entre comillas simples)	`'T'`

Declaración de las variables en C++

```cpp
string nombreCompleto  = "Ana Lucia Torres Vega";
int    edad            = 19;
double notaFinal       = 15.75;
string codigoMatricula = "202601234";
bool   aprobado        = true;
char   inicialApellido = 'T';
```

En pseudocódigo, las mismas declaraciones serían:

```
Definir nombreCompleto, codigoMatricula Como Cadena
Definir edad Como Entero
Definir notaFinal Como Real
Definir aprobado Como Logico
Definir inicialApellido Como Caracter
```

Programa completo para comprobarlo

Archivo: `codigo/ej01_tipos_de_datos.cpp`

```cpp
// ============================================================
// Ejercicio 1 - Tipos de datos y variables          (Nivel: Basico)
// Objetivo: elegir el tipo de dato C++ adecuado para cada dato
//           y declarar las variables correspondientes.
// ============================================================
// Nota: en los codigos evitamos tildes y la letra enie para que
// el texto se vea bien en cualquier consola (Windows, Linux, etc.).

#include <iostream>
#include <string>      // necesario para poder usar el tipo string
using namespace std;

int main() {
    // ---------- Declaracion + inicializacion ----------
    string nombreCompleto  = "Ana Lucia Torres Vega"; // texto (puede tener espacios)
    int    edad            = 19;                      // numero entero, sin decimales
    double notaFinal       = 15.75;                   // numero real (con decimales)
    string codigoMatricula = "202601234";             // es un CODIGO: se guarda como texto
    bool   aprobado        = true;                    // solo puede ser true o false
    char   inicialApellido = 'T';                     // UN solo caracter, comillas simples

    // ---------- Mostramos los valores para comprobar ----------
    cout << "Nombre completo : " << nombreCompleto << endl;
    cout << "Edad            : " << edad << endl;
    cout << "Nota final      : " << notaFinal << endl;
    cout << "Codigo matricula: " << codigoMatricula << endl;
    cout << "Aprobado        : " << aprobado << "   (cout muestra 1 = true, 0 = false)" << endl;
    cout << "Aprobado (texto): " << boolalpha << aprobado << endl;  // boolalpha -> true/false
    cout << "Inicial apellido: " << inicialApellido << endl;

    // ---------- Extra: bytes que ocupa cada tipo (sizeof) ----------
    cout << endl << "--- Bytes que ocupa cada tipo en memoria ---" << endl;
    cout << "int    : " << sizeof(int)    << " bytes" << endl;
    cout << "double : " << sizeof(double) << " bytes" << endl;
    cout << "char   : " << sizeof(char)   << " byte"  << endl;
    cout << "bool   : " << sizeof(bool)   << " byte"  << endl;

    return 0;
}
```

Ejecución real:

```
Nombre completo : Ana Lucia Torres Vega
Edad            : 19
Nota final      : 15.75
Codigo matricula: 202601234
Aprobado        : 1   (cout muestra 1 = true, 0 = false)
Aprobado (texto): true
Inicial apellido: T

--- Bytes que ocupa cada tipo en memoria ---
int    : 4 bytes
double : 8 bytes
char   : 1 byte
bool   : 1 byte
```

Observa que `cout` muestra un `bool` como `1` (verdadero) o `0` (falso). Para que muestre la palabra `true` / `false` se usa el manipulador `boolalpha`.

¿Por qué el código de matrícula NO es `int`?

Es la pregunta trampa del ejercicio. Aunque `202601234` «parece» un número (y hasta cabría en un `int`), es un **código**:

- No tiene sentido sumarlo, restarlo ni sacarle promedio.
- Podría empezar con ceros (`"001234"`) y en un `int` se perderían; peor aún, escrito como `int x = 001234;` C++ lo leería en **octal**.
- Lo normal es procesarlo por partes, por ejemplo, el año de ingreso: `codigoMatricula.substr(0, 4)` → `"2026"`. Eso solo se puede hacer con un `string`.

Regla general: DNI, teléfono, código de alumno, código de producto, número de cuenta → `string`.

Errores frecuentes

Código incorrecto	Qué pasa	Correcto
`int notaFinal = 15.75;`	Compila, pero guarda **15**: se pierden los decimales	`double notaFinal = 15.75;`
`char inicial = "T";`	**Error de compilación**: `"T"` es un texto, no un carácter	`char inicial = 'T';`
`string nombre = 'Ana';`	**Error**: comillas simples para varias letras	`string nombre = "Ana";`
`string codigo = 202601234;`	**Error de compilación**: un número no se convierte solo a `string`	`string codigo = "202601234";`
`bool aprobado = verdadero;`	**Error**: en C++ se escribe en inglés	`bool aprobado = true;`
Usar `string` sin `#include <string>`	Puede no compilar en algunos compiladores	Incluir `<string>` siempre

Practica: ¿qué tipo usarías?

Dato	Tipo	Dato	Tipo
DNI `"07654321"`	`string`	Precio de un producto	`double`
Cantidad de hijos	`int`	¿Tiene beca?	`bool`
Sexo (`'M'` / `'F'`)	`char`	Correo electrónico	`string`
Temperatura en °C	`double`	Año de nacimiento	`int`

Ejercicio 2 — Entrada, proceso y salida

Nivel: Básico · **Objetivo:** practicar la estructura básica de un programa.

Enunciado. Una empresa desea calcular el pago de un trabajador con los datos *Horas trabajadas: 8* y *Pago por hora: 25.50*. Se pide identificar entradas, proceso y salida, elaborar el pseudocódigo e implementar la solución en C++. Fórmula: **Pago = Horas × Tarifa**.

Análisis Entrada – Proceso – Salida

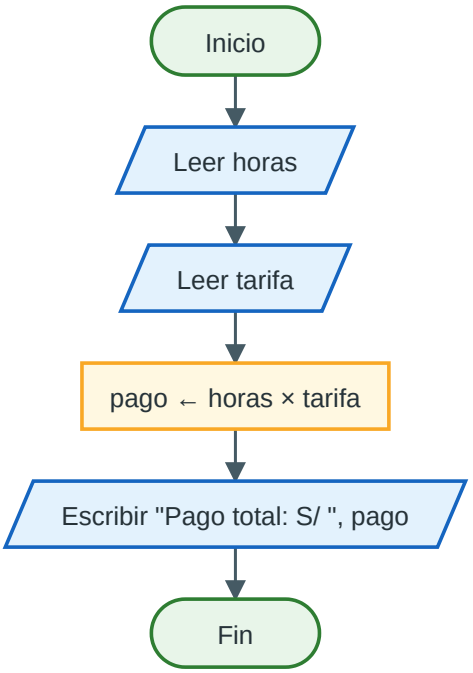
	Variable	Tipo	Justificación
Entrada	`horas`	`int`	Las horas trabajadas se cuentan en números enteros (8)
Entrada	`tarifa`	`double`	El pago por hora tiene decimales (25.50)
Proceso	`pago ← horas × tarifa`	—	La fórmula del enunciado
Salida	`pago`	`double`	`int × double` da un `double`; es dinero (2 decimales)

> **Nota**
>
> Si la empresa registrara fracciones de hora (por ejemplo, 7.5 horas), `horas` debería ser `double`. Con los datos del enunciado, `int` es la elección natural; cualquiera de las dos es válida **si la justificas**.

Pseudocódigo

```
Algoritmo CalcularPago
    Definir horas Como Entero
    Definir tarifa, pago Como Real
    Escribir "Horas trabajadas: "
    Leer horas
    Escribir "Pago por hora: "
    Leer tarifa
    pago ← horas * tarifa
    Escribir "Pago total: S/ ", pago
FinAlgoritmo
```

Diagrama de flujo



Ejercicio 2: diagrama de flujo

Código C++

Archivo: `codigo/ej02_pago_trabajador.cpp`

```cpp
// ============================================================
// Ejercicio 2 - Entrada, proceso y salida          (Nivel: Basico)
// Calcula el pago de un trabajador:  Pago = Horas x Tarifa
// ------------------------------------------------------------
// ENTRADA : horas trabajadas (int), pago por hora (double)
// PROCESO : pago = horas * tarifa
// SALIDA  : pago total (double, con 2 decimales)
// ============================================================

#include <iostream>
#include <iomanip>     // para fixed y setprecision (formato de decimales)
using namespace std;

int main() {
    // 1) Declaracion de variables
    int    horas;      // horas trabajadas (numero entero)
    double tarifa;     // pago por hora en soles (tiene decimales: 25.50)
    double pago;       // resultado: pago total

    // 2) ENTRADA: pedimos los datos al usuario
    cout << "Horas trabajadas: ";
    cin >> horas;
    cout << "Pago por hora: ";
    cin >> tarifa;

    // 3) PROCESO: aplicamos la formula
    pago = horas * tarifa;          // int * double -> el resultado es double

    // 4) SALIDA: mostramos el resultado con 2 decimales
    cout << fixed << setprecision(2);
    cout << "Pago total: S/ " << pago << endl;

    return 0;
}
```

Ejecución real (lo escrito después de cada pregunta lo tecleó el usuario):

```
Horas trabajadas: 8
Pago por hora: 25.50
Pago total: S/ 204.00
```

Prueba de escritorio

Paso	Instrucción	`horas`	`tarifa`	`pago`	Pantalla
1	Declaración	?	?	?	
2	`cin >> horas;`	8	?	?	`Horas trabajadas: 8`
3	`cin >> tarifa;`	8	25.50	?	`Pago por hora: 25.50`
4	`pago = horas * tarifa;`	8	25.50	204.00	
5	`cout << … << pago;`	8	25.50	204.00	`Pago total: S/ 204.00`

Verificación a mano: 8 × 25.50 = 204.00 ✔

Errores frecuentes

Error	Qué pasa
Calcular `pago` **antes** de leer los datos	Se multiplica «basura»: el resultado no tiene sentido. El orden es **leer → calcular → mostrar**
Declarar `pago` como `int`	Con 7 horas a 25.50 se guardaría 178 en lugar de 178.50
Olvidar `#include <iomanip>`	`setprecision` no compila
Escribir `25,50` (con coma) al ejecutar	`cin` lee solo `25` y se detiene en la coma: el programa muestra **S/ 200.00**. En la consola, los decimales van con **punto**
Escribir `7.5` cuando `horas` es `int`	`cin` guarda 7 y el `.5` sobrante lo toma la siguiente lectura: ¡el programa **ni siquiera pregunta** la tarifa y muestra S/ 3.50!

Así se ve el último error en una ejecución real:

```
Horas trabajadas: 7.5
Pago por hora: Pago total: S/ 3.50
```

Ejercicio 3 — Trazado de expresiones

Nivel: Medio · **Objetivo:** comprender la división entera y la conversión de tipos.

Enunciado. Analiza el código y responde: 1) ¿Cuál es el valor de `x`? 2) ¿Cuál es el valor de `y`? 3) ¿Por qué los resultados son diferentes? 4) ¿Qué función cumple `static_cast<double>`?

```cpp
int a = 10;
int b = 4;

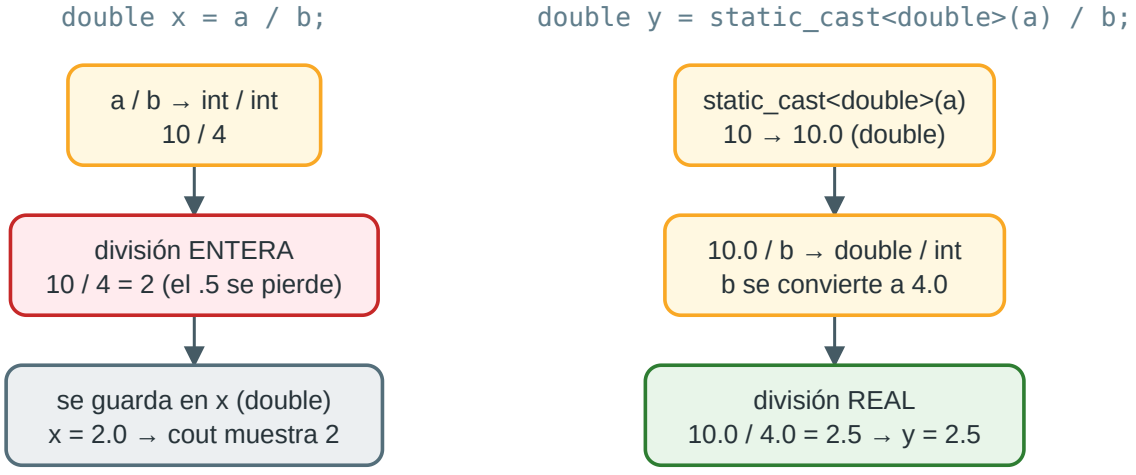
double x = a / b;
double y = static_cast<double>(a) / b;

cout << x << endl;
cout << y << endl;
```

Respuestas

1. `x` **vale 2** (guardado como `2.0`; en pantalla se ve `2`).
2. `y` **vale 2.5**.
3. **Porque en `x` se dividen dos enteros.** `a / b` es `int / int`, así que C++ hace **división entera**: 10 / 4 = 2 y el `.5` se descarta. Recién después ese `2` se convierte a `double` (2.0) para guardarlo en `x`; los decimales ya se habían perdido. En `y`, primero se convierte `a` a `double` (10.0), así que la operación es `double / int`; C++ convierte también `b` a 4.0 y hace **división real**: 10.0 / 4.0 = 2.5.
4. **`static_cast<double>` convierte explícitamente un valor al tipo `double`** para esa expresión. No cambia la variable `a` (sigue siendo `int` y sigue valiendo 10): crea una copia temporal de su valor como `double`. Sirve para forzar una división real entre variables enteras.

Paso a paso



Cómo evalúa C++ cada línea

Línea	Operación	Tipos	Resultado de la operación	Valor guardado
`double x = a / b;`	`10 / 4`	`int / int`	`2` (división entera)	`x = 2.0`
`double y = static_cast<double>(a) / b;`	`10.0 / 4`	`double / int` → `double / double`	`2.5` (división real)	`y = 2.5`

> **Nota**
>
> ¿Por qué se imprime `2` y no `2.0`? Porque `cout`, por defecto, **no muestra los ceros decimales innecesarios**. El valor guardado sí es 2.0. Con `cout << fixed << setprecision(1) << x;` se vería `2.0`.

Programa completo con variantes

Archivo: `codigo/ej03_division_entera.cpp`

```cpp
// ============================================================
// Ejercicio 3 - Trazado de expresiones              (Nivel: Medio)
// Objetivo: comprender la division entera y la conversion de tipos.
// ============================================================

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    // ---------- Codigo del enunciado ----------
    int a = 10;
    int b = 4;

    double x = a / b;                     // int / int -> division ENTERA: 10 / 4 = 2
                                          // (el .5 se pierde ANTES de guardar en x)
    double y = static_cast<double>(a) / b;  // double / int -> division REAL: 10.0 / 4 = 2.5

    cout << x << endl;    // muestra 2   (cout no muestra el ".0" por defecto)
    cout << y << endl;    // muestra 2.5

    // ---------- Extra: variantes que suelen aparecer en examen ----------
    cout << endl << "--- Variantes ---" << endl;
    cout << "a % b                      = " << a % b << endl;                      // 2 (resto)
    cout << "a / 4.0                    = " << a / 4.0 << endl;                    // 2.5
    cout << "static_cast<double>(a / b) = " << static_cast<double>(a / b) << endl;  // 2 (tarde!)
    cout << "1.0 * a / b                = " << 1.0 * a / b << endl;                // 2.5

    // Con fixed y setprecision(1) se ve el decimal aunque sea .0
    cout << fixed << setprecision(1);
    cout << "x con 1 decimal            = " << x << endl;                          // 2.0
    cout << "y con 1 decimal            = " << y << endl;                          // 2.5

    return 0;
}
```

Ejecución real:

```
2
2.5

--- Variantes ---
a % b                      = 2
a / 4.0                    = 2.5
static_cast<double>(a / b) = 2
1.0 * a / b                = 2.5
x con 1 decimal            = 2.0
y con 1 decimal            = 2.5
```

Trampas de examen

Trampa de examen

`static_cast<double>(a / b)` **no arregla nada**: primero se calcula `a / b` entre enteros (= 2) y recién después se convierte el 2 a 2.0. Hay que convertir **antes** de dividir: `static_cast<double>(a) / b`.

Expresión	Resultado	Explicación
`a / b`	`2`	entero / entero
`a % b`	`2`	resto de 10 entre 4
`a / 4.0`	`2.5`	`4.0` es `double` → división real
`1.0 * a / b`	`2.5`	`1.0 * a` ya es `double` (10.0)
`static_cast<double>(a / b)`	`2`	se convierte después de dividir: tarde
`int z = 7.9;`	`7`	al guardar un real en un `int` se cortan los decimales
`double w = 1 / 3;`	`0`	1 / 3 entre enteros = 0

Ejercicio 4 — Precedencia de operadores

Nivel: Medio · **Objetivo:** reforzar precedencia y asociatividad.

Enunciado. Con `int a = 8; int b = 3; int c = 2;` determina el resultado de: A) `a + b * c` B) `(a + b) * c` C) `a % b + c` D) `a / b * c`. Después, modifica cada expresión con paréntesis para dejar explícito el orden de evaluación.

Recordatorio de las reglas

1. Primero lo que está entre **paréntesis**.
2. Luego `*`, `/` y `%` (tienen **la misma** precedencia → se resuelven **de izquierda a derecha**).
3. Al final `+` y `-` (también de izquierda a derecha).
4. Como las tres variables son `int`, **toda división es entera**.

Solución paso a paso

A) `a + b * c`

- `*` tiene más precedencia que `+`: primero `b * c = 3 * 2 = 6`.
- Luego `a + 6 = 8 + 6 = 14`.
- **Resultado: 14** · Con paréntesis: `a + (b * c)`

B) `(a + b) * c`

- Los paréntesis obligan a sumar primero: `a + b = 8 + 3 = 11`.
- Luego `11 * c = 11 * 2 = 22`.
- **Resultado: 22** · Ya tiene los paréntesis necesarios: `(a + b) * c`

C) `a % b + c`

- `%` tiene más precedencia que `+`: primero `a % b = 8 % 3 = 2` (8 = 3 × 2 + **2**).
- Luego `2 + c = 2 + 2 = 4`.
- **Resultado: 4** · Con paréntesis: `(a % b) + c`

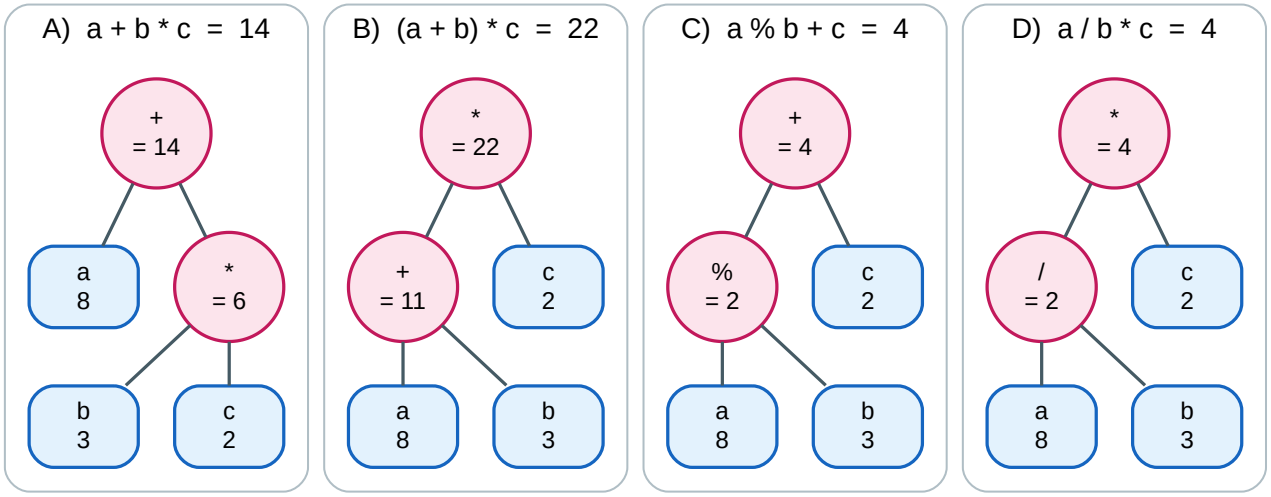
D) `a / b * c`

- `/` y `*` tienen la **misma** precedencia → se aplica la **asociatividad de izquierda a derecha**.
- Primero `a / b = 8 / 3 = 2` (división entera: 2.666… → 2).
- Luego `2 * c = 2 * 2 = 4`.
- **Resultado: 4** · Con paréntesis: `(a / b) * c`

Expresión	Orden explícito	Cálculo	Resultado
A) `a + b * c`	`a + (b * c)`	8 + 6	**14**
B) `(a + b) * c`	`(a + b) * c`	11 × 2	**22**
C) `a % b + c`	`(a % b) + c`	2 + 2	**4**
D) `a / b * c`	`(a / b) * c`	2 × 2	**4**

Árboles de evaluación

Cada expresión se puede dibujar como un árbol: **las hojas son los valores y se resuelve de abajo hacia arriba**. El operador que queda en la raíz es el **último** que se aplica.



Árboles de evaluación de las cuatro expresiones

¿Por qué importa la asociatividad? (expresión D)

Si alguien evaluara D de derecha a izquierda, haría `a / (b * c) = 8 / 6 = 1`, que es **otro resultado**. En C++, operadores de igual precedencia se evalúan de **izquierda a derecha**, por eso D da **4** y no 1.

Además, si fueran números reales: `8.0 / 3 * 2 = 5.33333` (no 4). En el ejercicio da 4 porque los tres son `int`.

Programa de comprobación

Archivo: `codigo/ej04_precedencia.cpp`

```cpp
// ============================================================
// Ejercicio 4 - Precedencia de operadores          (Nivel: Medio)
// Objetivo: reforzar precedencia y asociatividad.
// Regla: primero ( ), luego * / %  (de izquierda a derecha),
//        y al final + -            (de izquierda a derecha).
// ============================================================

#include <iostream>
using namespace std;

int main() {
    int a = 8;
    int b = 3;
    int c = 2;

    // ---------- Expresiones tal como las da el enunciado ----------
    cout << "A) a + b * c   = " << a + b * c   << endl;   // 8 + 6      = 14
    cout << "B) (a + b) * c = " << (a + b) * c << endl;   // 11 * 2     = 22
    cout << "C) a % b + c   = " << a % b + c   << endl;   // 2 + 2      = 4
    cout << "D) a / b * c   = " << a / b * c   << endl;   // 2 * 2      = 4

    // ---------- Las mismas, con parentesis explicitos ----------
    cout << endl;
    cout << "A) a + (b * c) = " << a + (b * c) << endl;   // 14
    cout << "B) (a + b) * c = " << (a + b) * c << endl;   // 22 (ya tenia parentesis)
    cout << "C) (a % b) + c = " << (a % b) + c << endl;   // 4
    cout << "D) (a / b) * c = " << (a / b) * c << endl;   // 4

    // ---------- Si agrupamos distinto, el resultado cambia ----------
    cout << endl;
    cout << "a / (b * c)    = " << a / (b * c) << endl;   // 8 / 6 = 1
    cout << "a % (b + c)    = " << a % (b + c) << endl;   // 8 % 5 = 3

    return 0;
}
```

Ejecución real:

```
A) a + b * c   = 14
B) (a + b) * c = 22
C) a % b + c   = 4
D) a / b * c   = 4

A) a + (b * c) = 14
B) (a + b) * c = 22
C) (a % b) + c = 4
D) (a / b) * c = 4

a / (b * c)    = 1
a % (b + c)    = 3
```

Practica más (con `a = 8, b = 3, c = 2`)

Expresión	Resultado	Por qué
`a - b - c`	`3`	De izquierda a derecha: (8 − 3) − 2
`a - (b - c)`	`7`	8 − 1
`a * b % c`	`0`	(8 × 3) % 2 = 24 % 2
`a + b % c`	`9`	8 + (3 % 2) = 8 + 1
`-a % b`	`-2`	(−8) % 3: el signo sigue al dividendo
`a / c / c`	`2`	(8 / 2) / 2

Ejercicio 5 — Debugging de entrada con string

Nivel: Medio · **Objetivo:** diferenciar `cin` y `getline()`.

Enunciado. Analiza el programa. Si el usuario escribe `Juan Carlos Perez`: 5) ¿Qué mostrará el programa? 6) ¿Cuál es el problema? 7) Corrige el código. 8) Explica por qué `getline()` es más apropiado en este caso. *(La numeración 5 a 8 es la de la hoja original).*

Archivo: `codigo/ej05a_original_con_error.cpp`

```cpp
// ============================================================
// Ejercicio 5 - Debugging de entrada con string     (Nivel: Medio)
// VERSION ORIGINAL (CON ERROR), tal como aparece en el enunciado.
// Si el usuario escribe: Juan Carlos Perez
// el programa muestra solo:  Cliente: Juan
// Motivo: "cin >>" deja de leer en el primer espacio en blanco.
// ============================================================

#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre;

    cout << "Ingrese nombre completo: ";
    cin >> nombre;                    // <-- PROBLEMA: lee solo hasta el primer espacio

    cout << "Cliente: " << nombre;
    return 0;
}
```

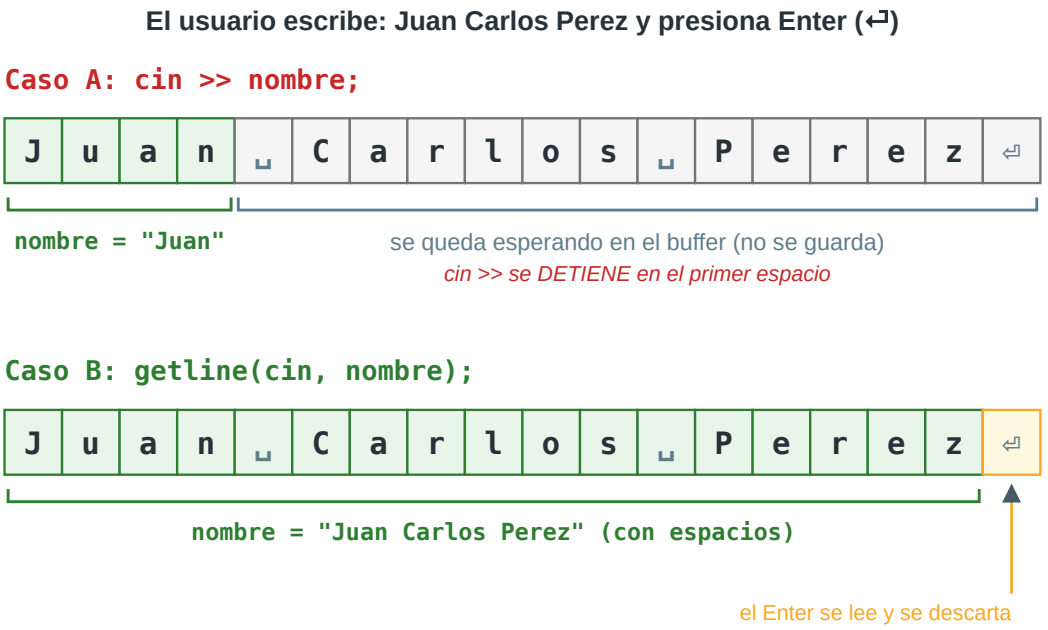
5) ¿Qué mostrará el programa?

```
Ingrese nombre completo: Juan Carlos Perez
Cliente: Juan
```

Solo muestra **`Cliente: Juan`**.

6) ¿Cuál es el problema?

`cin >> nombre` **lee únicamente hasta el primer espacio en blanco**. Para `cin >>`, el espacio es un separador de datos. Por eso guarda solo `"Juan"`; el resto (`" Carlos Perez"` y el Enter) **se queda esperando en el buffer de entrada** sin que nadie lo lea.



cin >> se detiene en el primer espacio; getline lee la línea completa

Lo que queda en el buffer no desaparece: si después hubiera otro `cin >>`, **lo tomaría sin esperar al usuario**. Ejecución real de un programa que, tras pedir el nombre con `cin >>`, pide la ciudad:

```
Ingrese nombre completo: Juan Carlos Perez
Ingrese ciudad: Cliente: Juan | Ciudad: Carlos
```

¡El programa nunca dejó escribir la ciudad: usó `"Carlos"`, que había quedado en el buffer!

7) Código corregido

Basta con cambiar **una línea**: `cin >> nombre;` por `getline(cin, nombre);`.

Archivo: `codigo/ej05b_corregido_getline.cpp`

```cpp
// ============================================================
// Ejercicio 5 - Debugging de entrada con string     (Nivel: Medio)
// VERSION CORREGIDA: usamos getline() para leer la linea completa.
// Si el usuario escribe: Juan Carlos Perez
// el programa muestra:   Cliente: Juan Carlos Perez
// ============================================================

#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre;

    cout << "Ingrese nombre completo: ";
    getline(cin, nombre);             // lee TODO hasta que el usuario presiona Enter

    cout << "Cliente: " << nombre << endl;
    return 0;
}
```

Ejecución real:

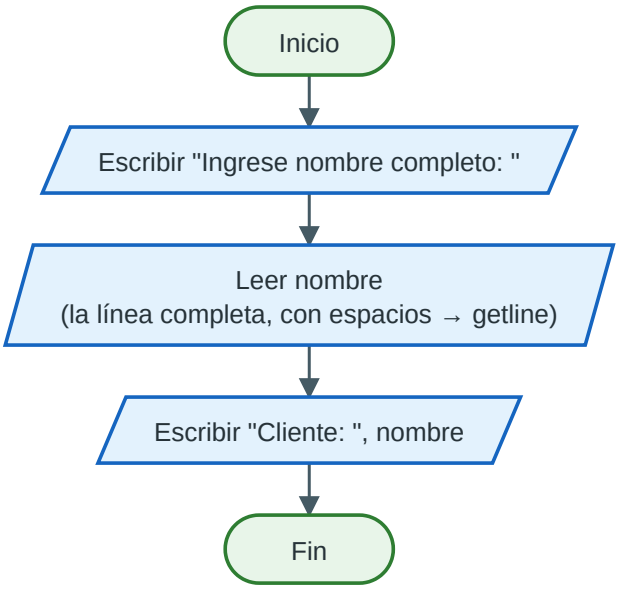
```
Ingrese nombre completo: Juan Carlos Perez
Cliente: Juan Carlos Perez
```

8) ¿Por qué `getline()` es más apropiado?

Porque un **nombre completo contiene espacios**. `getline(cin, nombre)` lee **toda la línea** —letras, espacios y símbolos— hasta que el usuario presiona Enter, y descarta ese Enter. En cambio, `cin >>` está pensado para leer datos separados por espacios (números o palabras sueltas). Regla práctica: **si el texto puede tener espacios, usa `getline`**.

Pseudocódigo y diagrama de flujo (versión corregida)

```
Algoritmo MostrarCliente
    Definir nombre Como Cadena
    Escribir "Ingrese nombre completo: "
    Leer nombre        // la línea completa, con espacios
    Escribir "Cliente: ", nombre
FinAlgoritmo
```



Ejercicio 5 corregido: diagrama de flujo

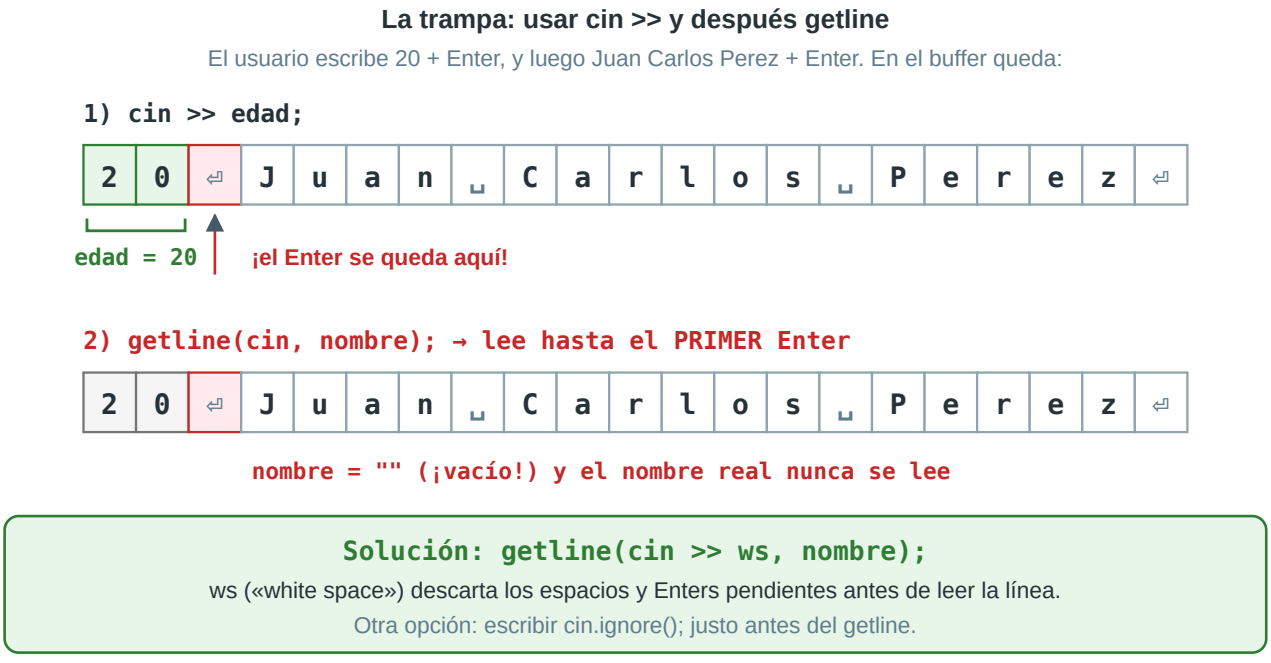
Extra: la trampa de mezclar `cin >>` con `getline`

En el examen es muy común que se pida primero un número y luego un texto. Si se lee el número con `cin >>` y después el texto con `getline`, **el texto sale vacío**, porque `getline` encuentra el Enter que dejó `cin >>`:

```
cout << "Cantidad: ";
cin >> cantidad;                // el usuario escribe 2 y Enter: el Enter queda pendiente
cout << "Nombre del cliente: ";
getline(cin, cliente);          // toma ese Enter: cliente queda VACIO
cout << "Cliente: [" << cliente << "]" << endl;
```

Ejecución real (el programa ni siquiera deja escribir el nombre):

```
Cantidad: 2
Nombre del cliente: Cliente: []
```



Después de cin >> queda un Enter en el buffer

La solución es descartar ese Enter con `ws` antes de leer la línea:

Archivo: `codigo/ej05c_extra_cin_y_getline.cpp`

```cpp
// ============================================================
// Ejercicio 5 - EXTRA: la trampa de mezclar "cin >>" con getline()
// ------------------------------------------------------------
// Despues de "cin >> edad;" el Enter (el caracter '\n') se queda
// esperando en el buffer de entrada. Un getline() que venga despues
// lo encuentra y "lee" una linea VACIA.
// Solucion: getline(cin >> ws, nombre);   (ws salta espacios y Enter)
//           o bien:  cin.ignore();  antes del getline.
// ============================================================

#include <iostream>
#include <string>
using namespace std;

int main() {
    int edad;
    string nombre;

    cout << "Ingrese su edad: ";
    cin >> edad;                      // lee el numero, pero deja el '\n' en el buffer

    cout << "Ingrese nombre completo: ";
    getline(cin >> ws, nombre);       // "cin >> ws" descarta el '\n' pendiente

    cout << "Cliente: " << nombre << " (" << edad << " anios)" << endl;
    return 0;
}
```

Ejecución real:

```
Ingrese su edad: 20
Ingrese nombre completo: Juan Carlos Perez
Cliente: Juan Carlos Perez (20 anios)
```

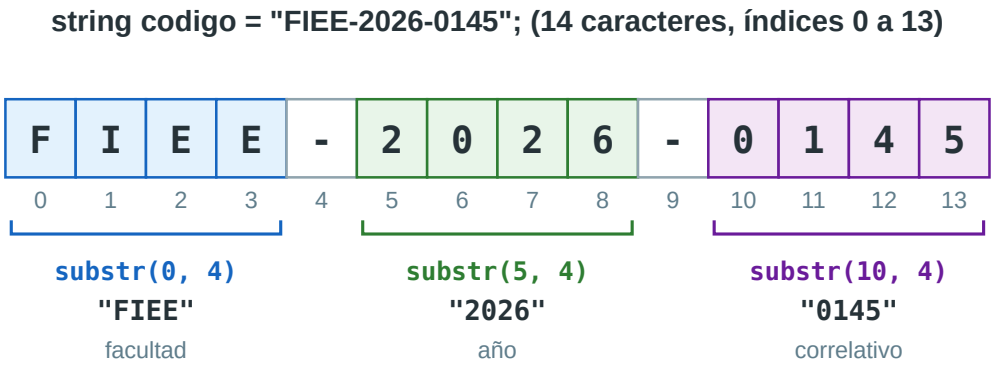
	`cin >> variable`	`getline(cin, variable)`
Lee hasta…	El primer espacio, tabulación o Enter	El Enter (fin de la línea)
¿Incluye espacios?	No	Sí
¿Qué pasa con el Enter?	Lo **deja** en el buffer	Lo **lee y lo descarta**
Ideal para	Números y palabras sueltas	Nombres, direcciones, frases
Tipos	`int`, `double`, `char`, `string`…	Solo `string`

Ejercicio 6 — Procesamiento de código con substr()

Nivel: Medio · **Objetivo:** trabajar índices y subcadenas.

Enunciado. Un código universitario tiene el formato `FIEE-2026-0145`, donde FIEE = facultad, 2026 = año y 0145 = correlativo. Desarrolla un programa que extraiga y muestre cada componente usando `substr()`. Salida esperada: `Facultad: FIEE`, `Año: 2026`, `Correlativo: 0145`. Además, responde: ¿qué devuelve `codigo.substr(5,4)`?

Clave: dibujar el texto con sus índices



Los guiones (índices 4 y 9) no se copian: por eso el año empieza en 5 y el correlativo en 10.

Índices del código y las tres subcadenas

- El texto tiene **14 caracteres**, con índices del **0 al 13**.
- Los guiones están en las posiciones **4** y **9**.
- Cada parte empieza **una posición después** de un guion.

Recuerda: `substr(inicio, cantidad)` → **desde** `inicio`, **toma** `cantidad` caracteres.

Parte	Empieza en	Cantidad	Instrucción	Resultado
Facultad	0	4	`codigo.substr(0, 4)`	`"FIEE"`
Año	5	4	`codigo.substr(5, 4)`	`"2026"`
Correlativo	10	4	`codigo.substr(10, 4)`	`"0145"`

Respuesta directa: ¿qué devuelve `codigo.substr(5,4)`?

Devuelve `"2026"`: los 4 caracteres que empiezan en el índice 5 (`'2'`, `'0'`, `'2'`, `'6'`). Es el **año**. Ojo: devuelve un **texto** (`string`), no un número.

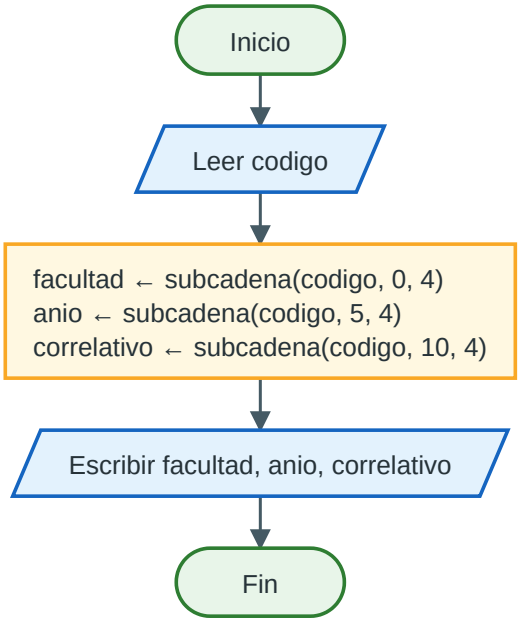
Análisis E-P-S

	Variable	Tipo	Justificación
Entrada	`codigo`	`string`	Tiene letras, números y guiones
Proceso	`facultad`, `anio`, `correlativo`	`string`	Son pedazos de un texto (`substr` devuelve `string`). El correlativo `"0145"` conserva su cero a la izquierda
Salida	las tres partes	—	Se muestran con su etiqueta

Pseudocódigo

```
Algoritmo ProcesarCodigo
    Definir codigo, facultad, anio, correlativo Como Cadena
    Escribir "Ingrese el codigo universitario: "
    Leer codigo
    facultad    ← subcadena(codigo, 0, 4)
    anio        ← subcadena(codigo, 5, 4)
    correlativo ← subcadena(codigo, 10, 4)
    Escribir "Facultad: ", facultad
    Escribir "Año: ", anio
    Escribir "Correlativo: ", correlativo
FinAlgoritmo
```

Diagrama de flujo



Ejercicio 6: diagrama de flujo

Código C++

Archivo: `codigo/ej06_substr_codigo.cpp`

```cpp
// ============================================================
// Ejercicio 6 - Procesamiento de codigo con substr() (Nivel: Medio)
// Codigo universitario con formato:  FIEE-2026-0145
//   FIEE = facultad, 2026 = anio, 0145 = correlativo
// ------------------------------------------------------------
// Indices de cada caracter (se empieza a contar desde 0):
//   F  I  E  E  -  2  0  2  6  -  0  1  4  5
//   0  1  2  3  4  5  6  7  8  9  10 11 12 13
// substr(inicio, cantidad) copia "cantidad" caracteres desde "inicio".
// ============================================================

#include <iostream>
#include <string>
using namespace std;

int main() {
    string codigo;

    // ENTRADA (el codigo no tiene espacios, por eso basta con cin >>)
    cout << "Ingrese el codigo universitario (ej. FIEE-2026-0145): ";
    cin >> codigo;

    // PROCESO: extraemos cada parte con substr(inicio, cantidad)
    string facultad    = codigo.substr(0, 4);    // desde el indice 0, 4 caracteres
    string anio        = codigo.substr(5, 4);    // desde el indice 5, 4 caracteres
    string correlativo = codigo.substr(10, 4);   // desde el indice 10, 4 caracteres

    // SALIDA
    cout << "Facultad: "    << facultad    << endl;
    cout << "Anio: "        << anio        << endl;   // en papel se escribe con enie
    cout << "Correlativo: " << correlativo << endl;

    // Pregunta del enunciado: que devuelve codigo.substr(5,4)?
    cout << "codigo.substr(5,4) devuelve: " << codigo.substr(5, 4) << endl;

    return 0;
}
```

Ejecución real:

```
Ingrese el codigo universitario (ej. FIEE-2026-0145): FIEE-2026-0145
Facultad: FIEE
Anio: 2026
Correlativo: 0145
codigo.substr(5,4) devuelve: 2026
```

> **Nota**
>
> Aquí se usó `cin >> codigo` porque el código **no tiene espacios**. Con `getline(cin, codigo)` también funcionaría. Y el mismo programa sirve para cualquier código con el mismo formato: con `FISI-2025-1234` muestra `FISI`, `2025` y `1234`.

Prueba de escritorio

Paso	Instrucción	`codigo`	`facultad`	`anio`	`correlativo`
1	`cin >> codigo;`	`"FIEE-2026-0145"`	?	?	?
2	`facultad = codigo.substr(0, 4);`	igual	`"FIEE"`	?	?
3	`anio = codigo.substr(5, 4);`	igual	`"FIEE"`	`"2026"`	?
4	`correlativo = codigo.substr(10, 4);`	igual	`"FIEE"`	`"2026"`	`"0145"`
5	Mostrar				Pantalla: `Facultad: FIEE`, `Anio: 2026`, `Correlativo: 0145`

Errores frecuentes

Escribir...	Da...	El error
`codigo.substr(5, 9)`	`"2026-0145"`	Pensar que el 2.º número es la posición **final**. ¡Es la **cantidad**!
`codigo.substr(4, 4)`	`"-202"`	Contar desde 1 en lugar de desde 0 (o incluir el guion)
`codigo.substr(10)`	`"0145"`	No es un error: sin 2.º número copia hasta el final. Correcto aquí porque el correlativo está al final
Guardar el correlativo en un `int`	`145`	Se pierde el cero a la izquierda

Para ir más allá

- **Últimos 4 caracteres**, sin contar posiciones: `codigo.substr(codigo.length() - 4)` → `"0145"`.
- **Convertir el año a número** para hacer cálculos: `stoi(anio) + 1` → `2027`.
- **Versión flexible con `find`** (funciona aunque la facultad tenga otra cantidad de letras):

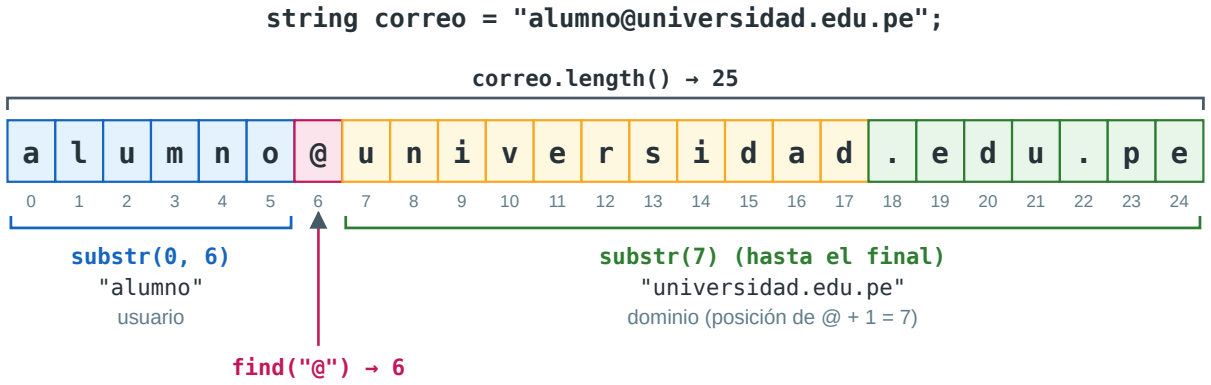
```
int g1 = codigo.find("-");              // primer guion: 4
int g2 = codigo.find("-", g1 + 1);      // segundo guion: 9 (busca desde g1 + 1)
string facultad    = codigo.substr(0, g1);              // "FIEE"
string anio        = codigo.substr(g1 + 1, g2 - g1 - 1); // "2026"
string correlativo = codigo.substr(g2 + 1);             // "0145"
```

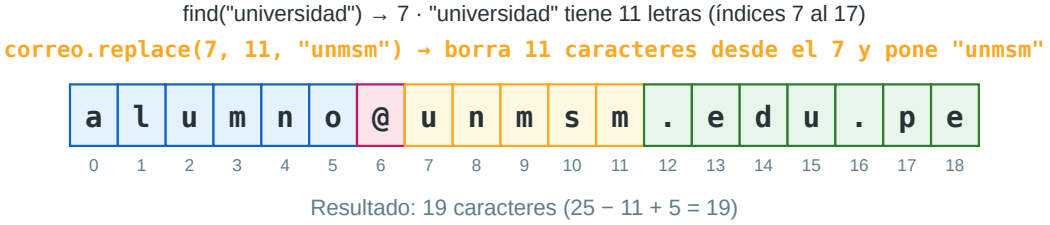
Ejercicio 7 — Búsqueda y reemplazo en strings

Nivel: Medio-alto · **Objetivo:** usar `length()`, `find()`, `substr()` y `replace()`.

Enunciado. Dada la cadena `string correo = "alumno@universidad.edu.pe";` 9) determina la longitud; 10) encuentra la posición de `@`; 11) extrae el nombre de usuario; 12) extrae el dominio; 13) reemplaza `"universidad"` por `"unmsm"`. Resultado esperado aproximado: `Usuario: alumno`, `Dominio: universidad.edu.pe`, `Nuevo correo: alumno@unmsm.edu.pe`.

El texto con sus índices





OJO: replace() MODIFICA la cadena original; si quieres conservarla, trabaja sobre una copia.

Índices del correo, búsqueda de la @ y reemplazo

Solución paso a paso

9) Longitud. `correo.length()` → **25**. Contando: `alumno` (6) + `@` (1) + `universidad` (11) + `.edu.pe` (7) = 25. Los índices van del 0 al 24.

10) Posición de `@`. `correo.find("@")` → **6**. Los índices 0 a 5 son `a-l-u-m-n-o`, así que la `@` está en el índice 6.

11) Usuario. Es todo lo que está **antes** de la `@`: `correo.substr(0, 6)` → **`"alumno"`**.

- ¿Por qué la cantidad es 6? Porque la `@` está en el índice 6, y **antes** de ella hay exactamente 6 caracteres (índices 0 a 5). En general: `correo.substr(0, posArroba)`.

12) Dominio. Es todo lo que está **después** de la `@`: empieza en `posArroba + 1 = 7` y va hasta el final: `correo.substr(7)` → **`"universidad.edu.pe"`** (18 caracteres).

13) Reemplazo. `replace(inicio, cantidad, nuevo)` borra `cantidad` caracteres desde `inicio` y pone `nuevo` en su lugar:

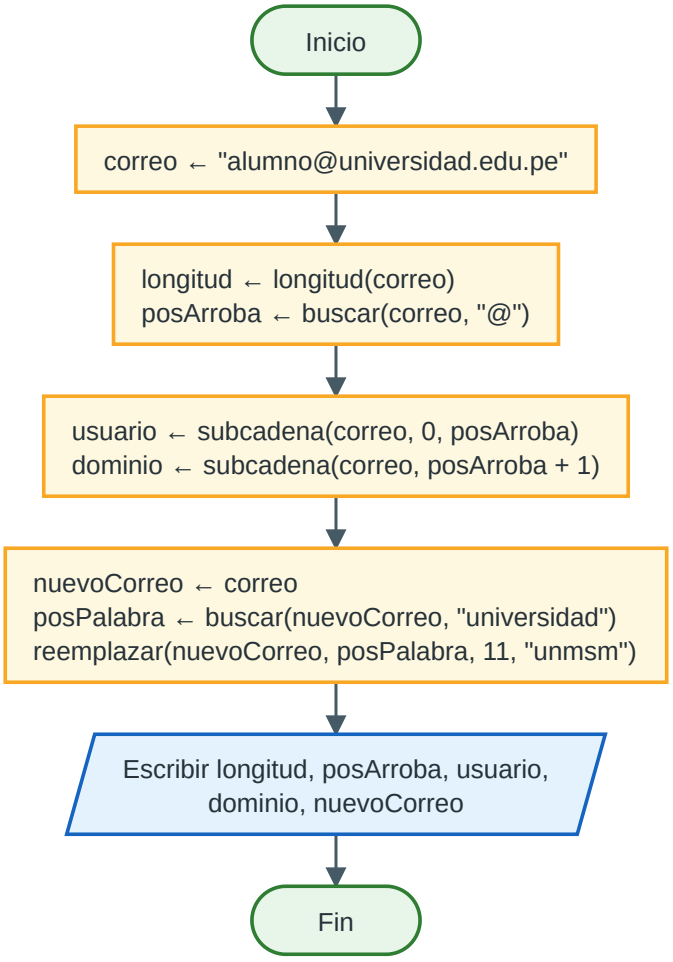
- `"universidad"` empieza en el índice **7** (`correo.find("universidad")` → 7) y tiene **11** letras.
- `correo.replace(7, 11, "unmsm")` → **`"alumno@unmsm.edu.pe"`** (19 caracteres: 25 − 11 + 5).

> **Cuidado**
>
> `replace` **modifica la variable original**. Si después necesitas el correo antiguo, primero haz una copia (`string nuevoCorreo = correo;`) y reemplaza en la copia. Así lo hace el programa de abajo.

Pseudocódigo

```
Algoritmo ProcesarCorreo
    Definir correo, usuario, dominio, nuevoCorreo Como Cadena
    Definir longitudCorreo, posArroba, posPalabra Como Entero
    correo ← "alumno@universidad.edu.pe"
    longitudCorreo ← longitud(correo)
    posArroba ← buscar(correo, "@")
    usuario ← subcadena(correo, 0, posArroba)
    dominio ← subcadena(correo, posArroba + 1, hasta el final)
    nuevoCorreo ← correo
    posPalabra ← buscar(nuevoCorreo, "universidad")
    nuevoCorreo ← reemplazar(nuevoCorreo, posPalabra, 11, "unmsm")
    Escribir "Longitud: ", longitudCorreo
    Escribir "Posicion de @: ", posArroba
    Escribir "Usuario: ", usuario
    Escribir "Dominio: ", dominio
    Escribir "Nuevo correo: ", nuevoCorreo
FinAlgoritmo
```

Diagrama de flujo



Ejercicio 7: diagrama de flujo

Código C++

Archivo: `codigo/ej07_correo.cpp`

```cpp
// ============================================================
// Ejercicio 7 - Busqueda y reemplazo en strings  (Nivel: Medio-alto)
// Objetivo: usar length(), find(), substr() y replace().
// ------------------------------------------------------------
//   a l u m n o @ u n i v  e  r  s  i  d  a  d  .  e  d  u  .  p  e
//   0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
// ============================================================

#include <iostream>
#include <string>
using namespace std;

int main() {
    string correo = "alumno@universidad.edu.pe";

    // 1) Longitud: cantidad total de caracteres
    int longitud = correo.length();                 // 25

    // 2) Posicion (indice) donde esta la @
    int posArroba = correo.find("@");               // 6

    // 3) Usuario: todo lo que esta ANTES de la @
    //    (desde el indice 0, tantos caracteres como indica posArroba)
    string usuario = correo.substr(0, posArroba);   // "alumno"

    // 4) Dominio: todo lo que esta DESPUES de la @
    //    (sin segundo parametro, substr copia hasta el final)
    string dominio = correo.substr(posArroba + 1);  // "universidad.edu.pe"

    // 5) Reemplazar "universidad" por "unmsm"
    //    replace(inicio, cuantos_borrar, texto_nuevo) MODIFICA la cadena,
    //    por eso trabajamos sobre una copia y conservamos el original.
    string nuevoCorreo = correo;
    int posPalabra = nuevoCorreo.find("universidad");  // 7
    nuevoCorreo.replace(posPalabra, 11, "unmsm");       // 11 = letras de "universidad"

    // SALIDA
    cout << "Correo original: " << correo      << endl;
    cout << "Longitud: "        << longitud    << endl;
    cout << "Posicion de @: "   << posArroba   << endl;
    cout << "Usuario: "         << usuario     << endl;
    cout << "Dominio: "         << dominio     << endl;
    cout << "Nuevo correo: "    << nuevoCorreo << endl;

    return 0;
}
```

Ejecución real:

```
Correo original: alumno@universidad.edu.pe
Longitud: 25
Posicion de @: 6
Usuario: alumno
Dominio: universidad.edu.pe
Nuevo correo: alumno@unmsm.edu.pe
```

Prueba de escritorio

Paso	Instrucción	Variable que cambia	Valor
1	`correo = "alumno@universidad.edu.pe"`	`correo`	`"alumno@universidad.edu.pe"`
2	`longitud = correo.length()`	`longitud`	`25`
3	`posArroba = correo.find("@")`	`posArroba`	`6`
4	`usuario = correo.substr(0, 6)`	`usuario`	`"alumno"`
5	`dominio = correo.substr(7)`	`dominio`	`"universidad.edu.pe"`
6	`nuevoCorreo = correo`	`nuevoCorreo`	`"alumno@universidad.edu.pe"`
7	`posPalabra = nuevoCorreo.find("universidad")`	`posPalabra`	`7`
8	`nuevoCorreo.replace(7, 11, "unmsm")`	`nuevoCorreo`	`"alumno@unmsm.edu.pe"`

Errores frecuentes

Código	Resultado	Problema
`correo.substr(posArroba)`	`"@universidad.edu.pe"`	Incluye la `@` : falta el `+ 1`
`correo.substr(0, posArroba + 1)`	`"alumno@"`	Se lleva la `@` con el usuario
`correo.replace(7, 5, "unmsm")`	`"alumno@unmsmrsidad.edu.pe"`	La cantidad debe ser la longitud de lo que se **borra** (11), no de lo que se pone (5)
`correo.replace("universidad", "unmsm")`	No compila	`replace` pide posición y cantidad, no el texto viejo

> **Consejo**
>
> Para no contar letras a mano: `string viejo = "universidad";` y luego `nuevoCorreo.replace(nuevoCorreo.find(viejo), viejo.length(), "unmsm");` . Así el programa funciona aunque cambie la palabra.

¿Funciona con otro correo?

Sí, porque usamos `find` en lugar de números fijos. Con `"maria.lopez@universidad.edu.pe"` el mismo programa obtiene `maria.lopez` , `universidad.edu.pe` y `maria.lopez@unmsm.edu.pe` . Solo el 6 y el 7 de los comentarios cambiarían.

Ejercicio 8 — Problema comercial integrado

Nivel: Medio-alto · **Objetivo:** integrar variables, expresiones y formato de salida.

Enunciado. Una tienda solicita: producto, precio unitario, cantidad y porcentaje de descuento. Debe calcular **Subtotal = Precio × Cantidad**, **Descuento = Subtotal × (Porcentaje / 100)** y **Total = Subtotal − Descuento**. Datos: Producto: Monitor, Precio: 850.50, Cantidad: 2, Descuento: 7.5. Se pide: 14) identificar Entrada–Proceso–Salida; 15) elaborar pseudocódigo; 16) escribir el programa en C++; 17) mostrar la salida con dos decimales.

14) Análisis Entrada – Proceso – Salida

	Variable	Tipo	Justificación
Entrada	`producto`	`string`	Es texto y puede tener espacios («Monitor LG 24») → `getline`
Entrada	`precio`	`double`	Dinero con decimales (850.50)
Entrada	`cantidad`	`int`	Unidades enteras (2)
Entrada	`porcentaje`	`double`	Puede tener decimales (7.5)
Proceso	`subtotal ← precio × cantidad`	`double`	
Proceso	`descuento ← subtotal × (porcentaje / 100)`	`double`	
Proceso	`total ← subtotal − descuento`	`double`	
Salida	`subtotal`, `descuento`, `total`	`double`	Con 2 decimales (`fixed` + `setprecision(2)`)

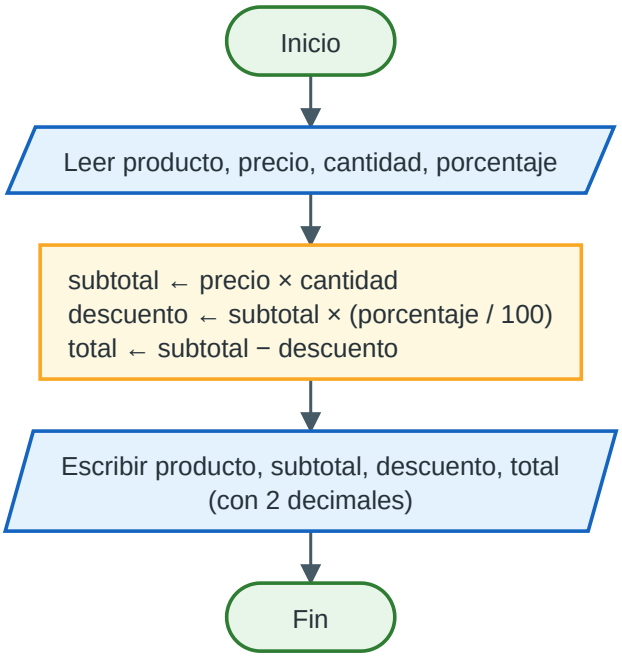
Trampa de examen

Si `porcentaje` fuera `int` (por ejemplo, 10), la expresión `porcentaje / 100` sería **división entera** y daría **0**: ¡el descuento siempre sería cero! Por eso `porcentaje` es `double` (o se escribe `porcentaje / 100.0`).

15) Pseudocódigo

```
Algoritmo VentaConDescuento
    Definir producto Como Cadena
    Definir precio, porcentaje, subtotal, descuento, total Como Real
    Definir cantidad Como Entero
    Escribir "Producto: "
    Leer producto
    Escribir "Precio: "
    Leer precio
    Escribir "Cantidad: "
    Leer cantidad
    Escribir "Descuento (%): "
    Leer porcentaje
    subtotal  ← precio * cantidad
    descuento ← subtotal * (porcentaje / 100)
    total     ← subtotal - descuento
    Escribir "Subtotal: ", subtotal          // con 2 decimales
    Escribir "Descuento: ", descuento
    Escribir "Total: ", total
FinAlgoritmo
```

Diagrama de flujo



Ejercicio 8: diagrama de flujo

16) Código C++

Archivo: `codigo/ej08_tienda_descuento.cpp`

```cpp
// ============================================================
// Ejercicio 8 - Problema comercial integrado      (Nivel: Medio-alto)
// ------------------------------------------------------------
// ENTRADA : producto (string), precio unitario (double),
//           cantidad (int), porcentaje de descuento (double)
// PROCESO : subtotal  = precio * cantidad
//           descuento = subtotal * (porcentaje / 100)
//           total     = subtotal - descuento
// SALIDA  : subtotal, descuento y total con 2 decimales
// ============================================================

#include <iostream>
#include <iomanip>     // fixed, setprecision
#include <string>      // string, getline
using namespace std;

int main() {
    // ---------- Declaracion de variables ----------
    string producto;       // nombre del producto (puede tener espacios)
    double precio;         // precio unitario (con decimales)
    int    cantidad;       // unidades compradas (entero)
    double porcentaje;     // % de descuento (puede ser 7.5)
    double subtotal, descuento, total;

    // ---------- ENTRADA ----------
    cout << "Producto: ";
    getline(cin, producto);        // primero el texto (getline), luego los numeros (cin)
    cout << "Precio: ";
    cin >> precio;
    cout << "Cantidad: ";
    cin >> cantidad;
    cout << "Descuento (%): ";
    cin >> porcentaje;
```

```
    // ---------- PROCESO ----------
    subtotal  = precio * cantidad;
    descuento = subtotal * (porcentaje / 100);   // 7.5 / 100 = 0.075
    total     = subtotal - descuento;

    // ---------- SALIDA ----------
    cout << fixed << setprecision(2);            // desde aqui: 2 decimales
    cout << endl;
    cout << "========= DETALLE DE LA COMPRA =========" << endl;
    cout << "Producto       : " << producto << endl;
    cout << "Precio unitario: S/ " << precio << endl;
    cout << "Cantidad       : " << cantidad << endl;
    cout << "Descuento      : " << porcentaje << " %" << endl;
    cout << "---------------------------------------" << endl;
    cout << "Subtotal       : S/ " << subtotal << endl;
    cout << "Monto descuento: S/ " << descuento << endl;
    cout << "TOTAL A PAGAR  : S/ " << total << endl;

    return 0;
}
```

17) Salida con dos decimales (ejecución real)

```
Producto: Monitor
Precio: 850.50
Cantidad: 2
Descuento (%): 7.5

========= DETALLE DE LA COMPRA =========
Producto       : Monitor
Precio unitario: S/ 850.50
Cantidad       : 2
Descuento      : 7.50 %
---------------------------------------
Subtotal       : S/ 1701.00
Monto descuento: S/ 127.57
TOTAL A PAGAR  : S/ 1573.42
```

Prueba de escritorio (con los valores exactos)

Paso	Instrucción	Cálculo	Resultado exacto
1	Leer datos	—	`producto = "Monitor"`, `precio = 850.50`, `cantidad = 2`, `porcentaje = 7.5`
2	`subtotal = precio * cantidad`	850.50 × 2	**1701.00**
3	`descuento = subtotal * (porcentaje / 100)`	1701.00 × 0.075	**127.575**
4	`total = subtotal - descuento`	1701.00 − 127.575	**1573.425**

¿Por qué sale 127.57 y no 127.58?

Si redondeamos a mano, 127.575 → 127.58 y 1573.425 → 1573.43. Pero el programa muestra **127.57** y **1573.42**. No es un error del programa:

1. La computadora guarda los `double` en **binario**, y muchos decimales (como 0.075) **no tienen una representación binaria exacta**, igual que 1/3 no tiene una representación decimal exacta (0.3333…).

2. Por eso, en memoria, el descuento queda como **127.57499999999998863...** y el total como **1573.42499999999995452...**: un poquito **menos** que el valor exacto.
3. `setprecision(2)` redondea **ese** valor guardado, y como está por debajo de la mitad, redondea hacia abajo.

Si en el examen te piden «la salida esperada», puedes escribir los valores de la prueba de escritorio (127.575 y 1573.425) y aclarar que con dos decimales el programa muestra 127.57 y 1573.42 por la representación binaria. En programas comerciales reales se trabaja con céntimos enteros o con tipos decimales especiales para evitar este efecto.

> **Nota**
>
> Este fenómeno es justamente el tema de «cómo se almacena un número real en la memoria» (estándar IEEE 754).

Errores frecuentes

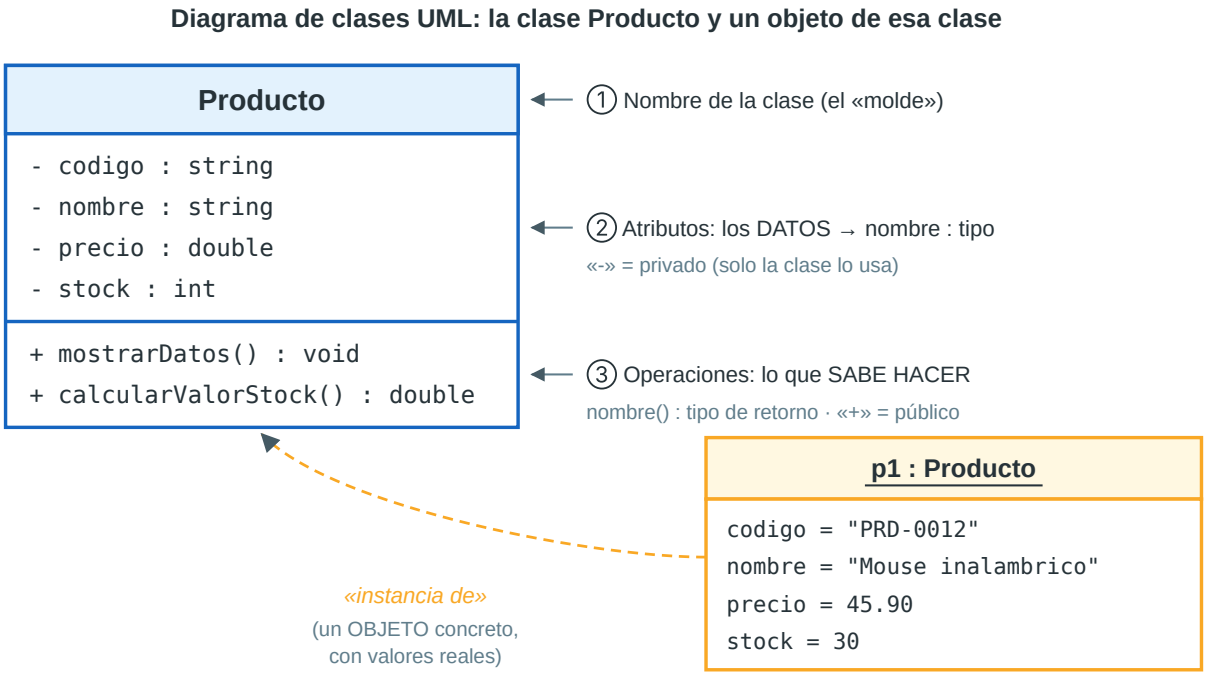
Error	Consecuencia
`int porcentaje;` y luego `porcentaje / 100`	Descuento = 0 para cualquier porcentaje menor que 100
Leer `producto` con `cin >>`	«Monitor LG 24» se corta en «Monitor» y el resto desordena las siguientes lecturas
Leer primero los números y luego el producto con `getline` (sin `ws`)	`producto` queda vacío (trampa del ejercicio 5)
Olvidar `fixed`	`setprecision(2)` significaría «2 cifras significativas»: `1701` se mostraría como `1.7e+03`
Escribir `cout << setprecision(2)` en cada línea	No es un error, pero es innecesario: el formato se queda activo

Ejercicio 9 — UML básico + C++

Nivel: Avanzado · **Objetivo:** relacionar el modelamiento UML con la implementación.

Enunciado. Se desea representar la entidad **Producto** con los datos código, nombre, precio y stock, y las operaciones `mostrarDatos()` y `calcularValorStock()`. Se pide: 18) construir un diagrama de clase UML básico; 19) identificar atributos y operaciones; 20) proponer los tipos C++ adecuados; 21) escribir una versión simplificada de la estructura en C++ (`string codigo; string nombre; double precio; int stock;`).

18) Diagrama de clases UML



Clase Producto y un objeto de ejemplo

En texto (como podrías dibujarlo en el examen):

```
┌──────────────────────────────────┐
│            Producto              │
├──────────────────────────────────┤
│ - codigo : string                │
│ - nombre : string                │
│ - precio : double                │
│ - stock : int                    │
├──────────────────────────────────┤
│ + mostrarDatos() : void          │
│ + calcularValorStock() : double  │
└──────────────────────────────────┘
```

19) Atributos y operaciones

Tipo de miembro	Nombre	Qué representa
Atributo	`codigo`	Identificador del producto (ej. `"PRD-0012"`)
Atributo	`nombre`	Descripción (ej. `"Mouse inalambrico"`)
Atributo	`precio`	Precio unitario en soles
Atributo	`stock`	Unidades disponibles en almacén
Operación	`mostrarDatos()`	Muestra en pantalla los datos del producto; **no devuelve** nada (`void`)
Operación	`calcularValorStock()`	Calcula y **devuelve** el valor total del inventario: `precio × stock` (`double`)

20) Tipos C++ propuestos

Miembro	Tipo	Justificación
`codigo`	`string`	Es un identificador: puede tener letras, guiones y ceros a la izquierda
`nombre`	`string`	Es texto
`precio`	`double`	Dinero con decimales
`stock`	`int`	Se cuentan unidades enteras
`mostrarDatos()`	`void`	Solo muestra; no devuelve un valor
`calcularValorStock()`	`double`	`double × int` da `double`

21) Versión simplificada en C++ (`struct`)

Es la «estructura» que pide el enunciado: agrupa los cuatro datos. Le añadimos las dos operaciones para que el programa esté completo.

Archivo: `codigo/ej09a_producto_struct.cpp`

```cpp
// ============================================================
// Ejercicio 9 - UML basico + C++                    (Nivel: Avanzado)
// VERSION SIMPLIFICADA (struct): la "estructura" que pide el enunciado.
// ------------------------------------------------------------
// Un struct agrupa varios datos relacionados bajo un mismo nombre.
// Aqui ademas le agregamos las dos operaciones del diagrama UML.
// ============================================================

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

struct Producto {
    // ---- Atributos (datos) ----
    string codigo;     // texto: puede tener letras y ceros a la izquierda
    string nombre;     // texto
    double precio;     // real: tiene decimales
    int    stock;      // entero: unidades disponibles

    // ---- Operaciones (funciones que pertenecen al struct) ----
    void mostrarDatos() {
        cout << "Codigo : " << codigo << endl;
        cout << "Nombre : " << nombre << endl;
        cout << "Precio : S/ " << precio << endl;
        cout << "Stock  : " << stock << " unidades" << endl;
    }

    double calcularValorStock() {
        return precio * stock;        // valor de todas las unidades en almacen
    }
};

int main() {
    cout << fixed << setprecision(2);

    Producto p;                       // p es un OBJETO (una variable) de tipo Producto
    p.codigo = "PRD-0012";            // con el punto (.) accedemos a cada atributo
    p.nombre = "Mouse inalambrico";
    p.precio = 45.90;
    p.stock  = 30;

    p.mostrarDatos();                 // llamamos a la operacion
    cout << "Valor del stock: S/ " << p.calcularValorStock() << endl;

    return 0;
}
```

Ejecución real:

```
Codigo : PRD-0012
Nombre : Mouse inalambrico
Precio : S/ 45.90
Stock  : 30 unidades
Valor del stock: S/ 1377.00
```

Versión completa: traducción fiel del UML (`class`)

En el diagrama, los atributos tienen `-` (privados) y las operaciones `+` (públicas). Eso se traduce con `private:` y `public:`. Como los atributos son privados, `main` no puede asignarlos directamente: se usa un **constructor**.

Archivo: `codigo/ej09b_producto_clase.cpp`

```cpp
// ================================================================
// Ejercicio 9 - UML basico + C++                    (Nivel: Avanzado)
// VERSION COMPLETA (class): traduccion fiel del diagrama UML.
// ----------------------------------------------------------------
//   UML                                 C++
//   - atributo  (privado)       ->      private:
//   + operacion (publica)       ->      public:
//   - precio : double           ->      double precio;
//   + calcularValorStock(): double ->   double calcularValorStock()
// ================================================================

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Producto {
private:                        // "-" en UML: solo la propia clase los usa
    string codigo;
    string nombre;
    double precio;
    int    stock;

public:                         // "+" en UML: se pueden usar desde main
    // Constructor: se ejecuta al crear el objeto y le da sus valores iniciales
    Producto(string c, string n, double p, int s) {
        codigo = c;
        nombre = n;
        precio = p;
        stock  = s;
    }

    void mostrarDatos() {
        cout << "Codigo : " << codigo << endl;
        cout << "Nombre : " << nombre << endl;
        cout << "Precio : S/ " << precio << endl;
        cout << "Stock  : " << stock << " unidades" << endl;
    }

    double calcularValorStock() {
        return precio * stock;
    }
};

int main() {
    cout << fixed << setprecision(2);

    // Creamos un objeto de la clase Producto (se llama al constructor)
    Producto p1("PRD-0012", "Mouse inalambrico", 45.90, 30);

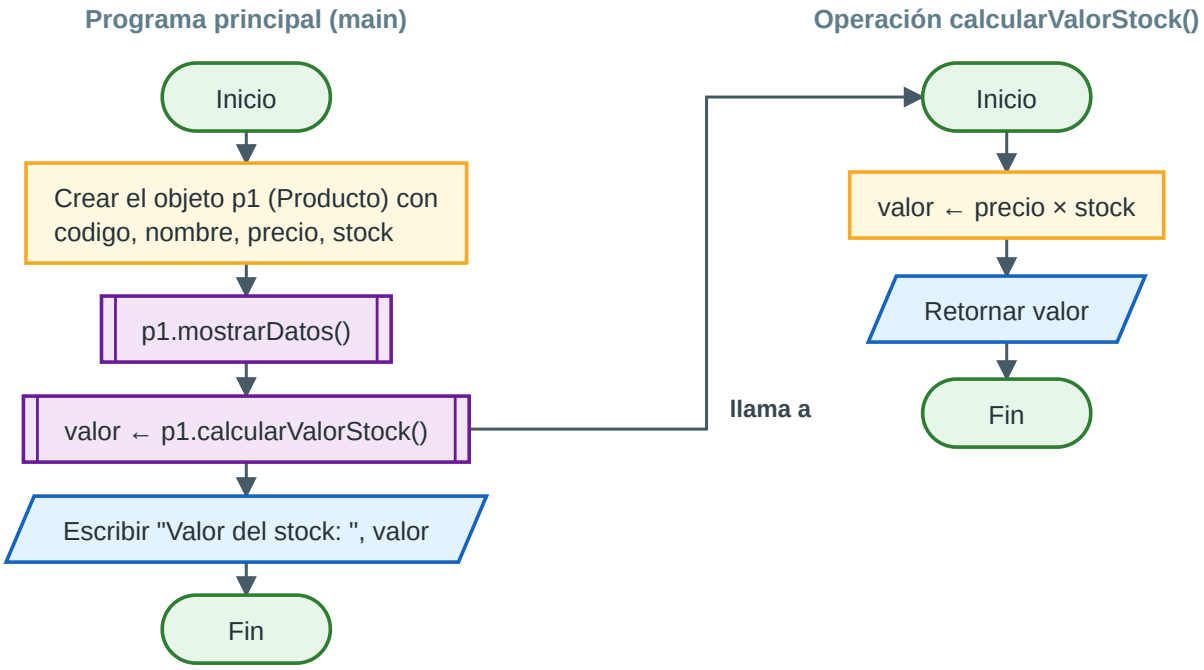
    p1.mostrarDatos();
    cout << "Valor del stock: S/ " << p1.calcularValorStock() << endl;

    // p1.precio = 10;   // ERROR de compilacion: precio es private

    return 0;
}
```

La ejecución muestra exactamente lo mismo que la versión `struct` (valor del stock: 45.90 × 30 = **S/ 1377.00**).

Diagrama de flujo



Ejercicio 9: programa principal y la operación calcularValorStock (el rectángulo con doble borde es la llamada a un subproceso o método)

Pseudocódigo de la operación:

```
Funcion valor ← calcularValorStock()
    valor ← precio * stock
FinFuncion
```

Correspondencia UML ↔ C++

En el diagrama	En el código
Rectángulo con el nombre `Producto`	`class Producto { … };`
`- codigo : string`	`private:` … `string codigo;`
`+ mostrarDatos() : void`	`public:` … `void mostrarDatos() { … }`
`+ calcularValorStock() : double`	`double calcularValorStock() { return precio * stock; }`
Objeto `p1 : Producto`	`Producto p1("PRD-0012", …);`

Errores frecuentes

- Olvidar el **punto y coma** después de la llave de cierre: `};`
- Llamar a un método **sin paréntesis**: `p1.mostrarDatos;` da **error de compilación** (*invalid use of non-static member function*). Debe ser `p1.mostrarDatos();`
- Intentar `p1.precio = 10;` cuando `precio` es `private` → **error de compilación** (es lo que el UML quiere proteger).
- Poner `int` como tipo de retorno de `calcularValorStock()`: se perderían los decimales.

Ejercicio 10 — Caso integrador de preparación

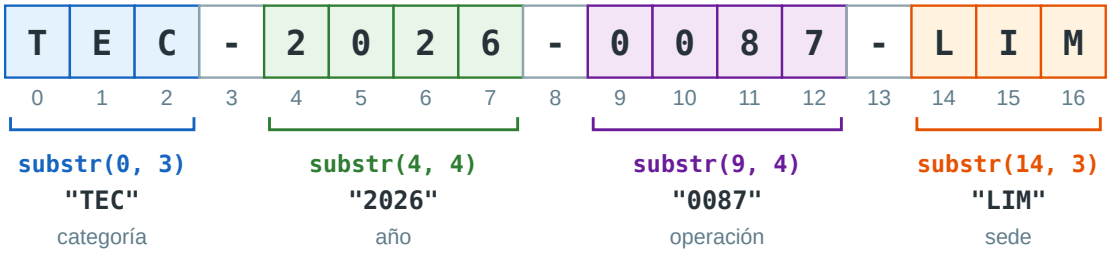
Nivel: Avanzado · **Objetivo:** integrar diseño, strings, expresiones y programación básica en C++.

Enunciado. Una tienda registra una venta con un código de estructura `TEC-2026-0087-LIM` (TEC = categoría, 2026 = año, 0087 = operación, LIM = sede). El programa recibe además: nombre del cliente, nombre del producto, precio unitario, cantidad y porcentaje de descuento.

- **Parte A** — Procesamiento de la cadena: obtener categoría, año, número de operación y sede con `substr()`.
- **Parte B** — Cálculos: Subtotal = Precio × Cantidad; MontoDescuento = Subtotal × (PorcentajeDescuento / 100); Total = Subtotal − MontoDescuento.
- **Parte C** — Salida: un «RESUMEN DE VENTA» con todos los datos.
- **Restricción:** sin `if`, `switch`, `for`, `while` ni `do-while`.
- **Entregables:** 22) Entrada–Proceso–Salida; 23) Pseudocódigo; 24) Código C++; 25) Prueba de escritorio; 26) Explicación breve de `substr()`.

El código de venta con sus índices

string codigoVenta = "TEC-2026-0087-LIM"; (17 caracteres, índices 0 a 16)



Regla rápida: cada parte empieza 1 posición después del guion anterior.

Índices del código de venta y sus cuatro partes

Parte	Empieza en	Cantidad	Instrucción	Resultado
Categoría	0	3	`codigoVenta.substr(0, 3)`	`"TEC"`
Año	4	4	`codigoVenta.substr(4, 4)`	`"2026"`
Operación	9	4	`codigoVenta.substr(9, 4)`	`"0087"`
Sede	14	3	`codigoVenta.substr(14, 3)`	`"LIM"`

22) Entrada – Proceso – Salida

	Variable	Tipo	Justificación
Entrada	`codigoVenta`	`string`	Letras, números y guiones
Entrada	`cliente`	`string`	Nombre con espacios → `getline`
Entrada	`producto`	`string`	Nombre con espacios → `getline`
Entrada	`precioUnitario`	`double`	Dinero con decimales
Entrada	`cantidad`	`int`	Unidades enteras
Entrada	`porcentajeDescuento`	`double`	Puede tener decimales; evita la división entera
Proceso A	`categoria`, `anio`, `operacion`, `sede`	`string`	Subcadenas del código (`substr` devuelve `string`; `"0087"` conserva sus ceros)
Proceso B	`subtotal`, `montoDescuento`, `total`	`double`	Resultados de operaciones con `double`
Salida	Todo lo anterior	—	Resumen con 2 decimales en los montos

23) Pseudocódigo

```
Algoritmo ResumenDeVenta
    Definir codigoVenta, cliente, producto Como Cadena
    Definir categoria, anio, operacion, sede Como Cadena
    Definir precioUnitario, porcentajeDescuento Como Real
    Definir subtotal, montoDescuento, total Como Real
    Definir cantidad Como Entero

    // ENTRADA
    Leer codigoVenta
    Leer cliente
    Leer producto
    Leer precioUnitario
    Leer cantidad
    Leer porcentajeDescuento

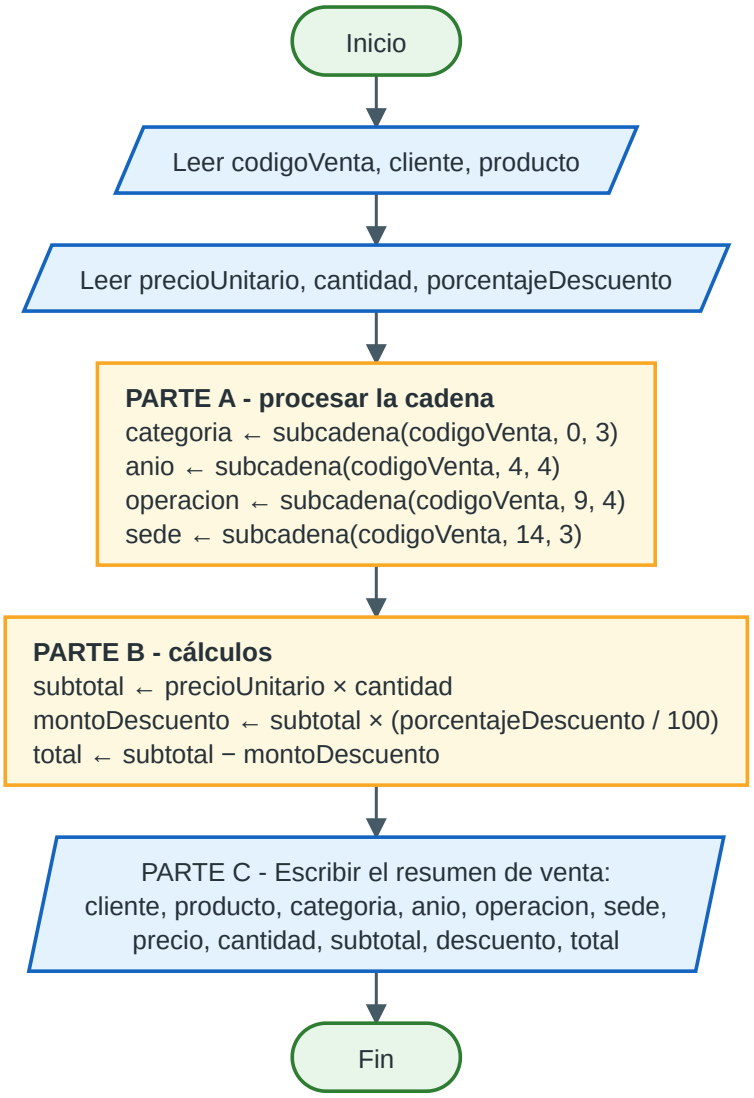
    // PARTE A: procesamiento de la cadena
    categoria ← subcadena(codigoVenta, 0, 3)
    anio      ← subcadena(codigoVenta, 4, 4)
    operacion ← subcadena(codigoVenta, 9, 4)
    sede      ← subcadena(codigoVenta, 14, 3)

    // PARTE B: calculos
    subtotal       ← precioUnitario * cantidad
    montoDescuento ← subtotal * (porcentajeDescuento / 100)
    total          ← subtotal - montoDescuento

    // PARTE C: salida
    Escribir "============ RESUMEN DE VENTA ============"
    Escribir "Cliente: ", cliente
    Escribir "Producto: ", producto
    Escribir "Categoría: ", categoria
    Escribir "Año: ", anio
    Escribir "Operación: ", operacion
    Escribir "Sede: ", sede
    Escribir "Precio unitario: ", precioUnitario
    Escribir "Cantidad: ", cantidad
    Escribir "Subtotal: ", subtotal
    Escribir "Descuento: ", montoDescuento
    Escribir "TOTAL: ", total
FinAlgoritmo
```

Diagrama de flujo

Es un diagrama **solo secuencial**: no hay rombos de decisión ni flechas que regresen, como exige la restricción.



Ejercicio 10: diagrama de flujo

24) Código C++

Archivo: `codigo/ej10_caso_integrador.cpp`

```cpp
// ============================================================
// Ejercicio 10 - Caso integrador de preparacion     (Nivel: Avanzado)
// Restriccion: SIN if, switch, for, while ni do-while (solo secuencia).
// ------------------------------------------------------------
// Codigo de venta:  TEC-2026-0087-LIM
//   T  E  C  -  2  0  2  6  -  0  0  8  7  -  L  I  M
//   0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16
//   categoria = substr(0, 3)   -> "TEC"
//   anio      = substr(4, 4)   -> "2026"
//   operacion = substr(9, 4)   -> "0087"
//   sede      = substr(14, 3)  -> "LIM"
// ============================================================

#include <iostream>
#include <iomanip>     // fixed, setprecision
#include <string>      // string, getline, substr
```

```cpp
using namespace std;

int main() {
    // ================= DECLARACION DE VARIABLES =================
    string codigoVenta, cliente, producto;
    double precioUnitario, porcentajeDescuento;
    int    cantidad;

    // ======================== ENTRADA ============================
    // Primero leemos TODOS los textos con getline y al final los numeros
    // con cin >>. Asi evitamos la trampa del '\n' que deja cin >>.
    cout << "Codigo de venta (ej. TEC-2026-0087-LIM): ";
    getline(cin, codigoVenta);
    cout << "Nombre del cliente: ";
    getline(cin, cliente);
    cout << "Nombre del producto: ";
    getline(cin, producto);
    cout << "Precio unitario: ";
    cin >> precioUnitario;
    cout << "Cantidad: ";
    cin >> cantidad;
    cout << "Porcentaje de descuento: ";
    cin >> porcentajeDescuento;

    // ======================== PROCESO ============================
    // Parte A - Procesamiento de la cadena con substr(inicio, cantidad)
    string categoria = codigoVenta.substr(0, 3);
    string anio      = codigoVenta.substr(4, 4);
    string operacion = codigoVenta.substr(9, 4);
    string sede      = codigoVenta.substr(14, 3);

    // Parte B - Calculos
    double subtotal       = precioUnitario * cantidad;
    double montoDescuento = subtotal * (porcentajeDescuento / 100);
    double total          = subtotal - montoDescuento;

    // ======================== SALIDA =============================
    // Parte C - Resumen (tildes omitidas a proposito: ver nota del codigo 1)
    cout << fixed << setprecision(2);
    cout << endl;
    cout << "============ RESUMEN DE VENTA ============" << endl;
    cout << endl;
    cout << "Cliente:         " << cliente << endl;
    cout << "Producto:        " << producto << endl;
    cout << endl;
    cout << "Categoria:       " << categoria << endl;
    cout << "Anio:            " << anio << endl;
    cout << "Operacion:       " << operacion << endl;
    cout << "Sede:            " << sede << endl;
    cout << endl;
    cout << "Precio unitario: S/ " << precioUnitario << endl;
    cout << "Cantidad:        " << cantidad << endl;
    cout << "Subtotal:        S/ " << subtotal << endl;
    cout << "Descuento:       S/ " << montoDescuento << endl;
    cout << "TOTAL:           S/ " << total << endl;

    return 0;
}
```

Ejecución real con datos de prueba:

```
Codigo de venta (ej. TEC-2026-0087-LIM): TEC-2026-0087-LIM
Nombre del cliente: Rosa Quispe Mamani
Nombre del producto: Laptop Lenovo IdeaPad 3
Precio unitario: 2450.00
Cantidad: 2
Porcentaje de descuento: 10

============ RESUMEN DE VENTA ============

Cliente:         Rosa Quispe Mamani
Producto:        Laptop Lenovo IdeaPad 3

Categoria:       TEC
Anio:            2026
Operacion:       0087
Sede:            LIM

Precio unitario: S/ 2450.00
Cantidad:        2
Subtotal:        S/ 4900.00
Descuento:       S/ 490.00
TOTAL:           S/ 4410.00
```

25) Prueba de escritorio

Datos de prueba: `codigoVenta = "TEC-2026-0087-LIM"`, `cliente = "Rosa Quispe Mamani"`, `producto = "Laptop Lenovo IdeaPad 3"`, `precioUnitario = 2450.00`, `cantidad = 2`, `porcentajeDescuento = 10`.

Paso	Instrucción	Variable	Valor
1	`getline(cin, codigoVenta)`	`codigoVenta`	`"TEC-2026-0087-LIM"`
2	`getline(cin, cliente)`	`cliente`	`"Rosa Quispe Mamani"`
3	`getline(cin, producto)`	`producto`	`"Laptop Lenovo IdeaPad 3"`
4	`cin >> precioUnitario`	`precioUnitario`	`2450.00`
5	`cin >> cantidad`	`cantidad`	`2`
6	`cin >> porcentajeDescuento`	`porcentajeDescuento`	`10`
7	`categoria = codigoVenta.substr(0, 3)`	`categoria`	`"TEC"`
8	`anio = codigoVenta.substr(4, 4)`	`anio`	`"2026"`
9	`operacion = codigoVenta.substr(9, 4)`	`operacion`	`"0087"`
10	`sede = codigoVenta.substr(14, 3)`	`sede`	`"LIM"`
11	`subtotal = 2450.00 * 2`	`subtotal`	`4900.00`
12	`montoDescuento = 4900.00 * (10 / 100)`	`montoDescuento`	`490.00`
13	`total = 4900.00 - 490.00`	`total`	`4410.00`
14	`cout << …`	Pantalla	El resumen de venta mostrado arriba

> **Importante**
>
> En el paso 12, `10 / 100` da **0.1** (y no 0) **porque `porcentajeDescuento es double`**. Si lo hubiéramos declarado `int`, la división sería entera, el descuento saldría 0.00 y el total 4900.00. Este detalle suele valer puntos en el examen.

26) Explicación breve de `substr()`

`substr` es una función de la clase `string` que **devuelve una copia de una parte del texto**. Se escribe `texto.substr(inicio, cantidad)`:

- `inicio` es el **índice** (posición, contando desde 0) donde empieza la parte que queremos;
- `cantidad` es **cuántos caracteres** se copian a partir de ahí;
- si se omite `cantidad`, copia **hasta el final**;
- **no modifica** el texto original y devuelve un nuevo `string`.

Ejemplo: en `"TEC-2026-0087-LIM"`, `substr(9, 4)` empieza en el índice 9 y copia 4 caracteres → `"0087"`.

Por qué esta solución respeta la restricción

No usa `if`, `switch` ni bucles: es una **secuencia** de instrucciones (leer → procesar → mostrar). Funciona porque el formato del código es **fijo** (siempre 3 letras, guion, 4 dígitos, guion, 4 dígitos, guion, 3 letras), así que las posiciones se conocen de antemano.

Errores frecuentes

Error	Consecuencia
Leer `codigoVenta` con `cin >>` y luego `cliente` con `getline` (sin `ws`)	`cliente` queda vacío y el nombre real se mezcla con lo siguiente
Usar `cin >> cliente`	Solo se guarda el primer nombre («Rosa»)
`substr(9, 13)` para la operación	Se piensa en «hasta el índice 13», pero el 2.° número es la **cantidad**: daría `"0087-LIM"`
`int porcentajeDescuento`	`10 / 100 = 0` → sin descuento
Olvidar `fixed << setprecision(2)`	`4410` sin decimales, o en notación científica en montos grandes

Parte IV — Repaso final

17. Checklist antes del parcial (respondido)

La hoja de ejercicios termina con una lista de comprobación. Aquí está **cada punto con su respuesta corta** y dónde repasarlo.

1. Puedo diferenciar algoritmo, código fuente y programa ejecutable. *Algoritmo*: los pasos para resolver el problema (independiente del lenguaje). *Código fuente*: el algoritmo escrito en C++ (archivo `.cpp`, texto legible). *Ejecutable*: el código traducido a lenguaje máquina por el compilador (archivo `.exe`). → Parte I, sección 2.

2. Puedo identificar entradas, procesos y salidas de un problema. *Entrada*: los datos que el programa no conoce y debe pedir. *Proceso*: los cálculos o transformaciones (fórmulas, `substr`, etc.). *Salida*: lo que se debe mostrar. Preséntalo en una tabla con variable, tipo y justificación. → Sección 4.1 y ejercicios 2, 8 y 10.

3. Puedo elegir tipos de datos apropiados en C++. Texto → `string`; un carácter → `char`; entero → `int`; con decimales → `double`; sí/no → `bool`. Códigos, DNI y teléfonos → `string`. → Secciones 10.4 y 10.5, ejercicio 1.

4. Comprendo la diferencia entre `cin` y `getline()`. `cin >>` lee hasta el primer espacio y deja el Enter en el buffer; sirve para números y palabras sueltas. `getline(cin, s)` lee la línea completa con espacios y consume el Enter; sirve para nombres. Si un `getline` viene después de un `cin >>`, usa `getline(cin >> ws, s)`. → Sección 11, ejercicio 5.

5. Puedo usar `length()`, `find()`, `substr()` y `replace()`. `length()` → cantidad de caracteres. `find(t)` → posición de la primera aparición (o `string::npos`). `substr(inicio, cantidad)` → copia una parte. `replace(inicio, cantidad, nuevo)` → cambia una parte (modifica el original). Los índices empiezan en 0. → Sección 15, ejercicios 6, 7 y 10.

6. Comprendo división entera, conversión de tipos y precedencia. `int / int` corta los decimales (`10 / 4 = 2`). Para división real, convierte **antes** de dividir: `static_cast<double>(a) / b`. Precedencia: `( )` → `* / %` → `+ -`; a igual precedencia, de izquierda a derecha. → Secciones 12 y 13, ejercicios 3 y 4.

7. Puedo leer y corregir fragmentos sencillos de código. Revisa siempre: ¿falta algún `;` o `#include`? ¿Cada variable tiene el tipo correcto? ¿Se lee con `cin` un texto con espacios? ¿Hay una división entre enteros? ¿Los índices de `substr` empiezan en 0 y el segundo número es una cantidad? ¿Se usa una variable antes de darle valor? → Ejercicios 3, 5 y la lista de errores frecuentes de abajo.

8. Puedo elaborar pseudocódigo antes de programar. Plantilla: `Algoritmo Nombre` → `Definir … Como …` → `Leer …` → asignaciones con `←` → `Escribir …` → `FinAlgoritmo`. Cada línea luego se traduce a una línea de C++. → Sección 5.

9. Puedo interpretar un diagrama de clase UML básico. Tres compartimentos: nombre / atributos (`- nombre : tipo`) / operaciones (`+ nombre() : retorno`). `-` = privado, `+` = público, `void` = no devuelve nada. → Sección 8, ejercicio 9.

10. Puedo desarrollar un programa integrador básico en C++. Receta: 1) tabla E-P-S con tipos; 2) dibuja los índices de los textos; 3) lee primero los textos con `getline` y después los números con `cin`; 4) extrae con `substr`; 5) calcula con `double`; 6) muestra con `fixed << setprecision(2)`; 7) haz la prueba de escritorio. → Ejercicio 10.

18. Los 15 errores más frecuentes

#	Error	Cómo evitarlo
1	Olvidar el `;` al final de una instrucción	Revisa cada línea antes de compilar
2	Escribir `Cout`, `Cin`, `Main` con mayúscula	C++ distingue mayúsculas: `cout`, `cin`, `main`
3	Usar `string` / `getline` sin `#include <string>`, o `setprecision` sin `#include <iomanip>`	Usa la plantilla de la sección 9
4	Dividir enteros esperando decimales (`10 / 4`)	Convierte antes: `static_cast<double>(10) / 4` o usa `4.0`
5	`static_cast<double>(a / b)`	El cast va **antes** de dividir: `static_cast<double>(a) / b`
6	`porcentaje / 100` con `porcentaje` entero	Declara `double porcentaje` o divide entre `100.0`
7	Leer un nombre completo con `cin >>`	Usa `getline(cin, nombre)`
8	`getline` después de `cin >>` (lee vacío)	`getline(cin >> ws, texto)` o lee primero los textos
9	Contar los índices desde 1	Los índices empiezan en **0**; dibuja el texto con sus índices
10	Usar el 2.° parámetro de `substr` como posición final	Es la **cantidad** de caracteres
11	Olvidar el `+ 1` al extraer lo que está después de un separador	`s.substr(pos + 1)`
12	`replace` con la cantidad del texto nuevo en vez del viejo	`replace(inicio, largoDelViejo, nuevo)`
13	`char c = "A";` o `string s = 'Hola';`	Simples para `char`, dobles para texto
14	Guardar códigos o DNI como `int`	Usa `string` (conserva ceros y permite `substr`)
15	Calcular antes de leer los datos	Orden: declarar → leer → calcular → mostrar

19. Hoja resumen

Esqueleto de un programa

```cpp
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

int main() {
    // declarar → leer → calcular → mostrar
    return 0;
}
```

Tipos de datos

Dato	Tipo	Ejemplo
Entero	`int`	`int edad = 19;`
Con decimales	`double`	`double precio = 850.50;`
Un carácter	`char`	`char inicial = 'T';`
Verdadero/falso	`bool`	`bool aprobado = true;`
Texto / códigos	`string`	`string codigo = "FIEE-2026-0145";`

Entrada y salida

Quiero...	Código
Mostrar	`cout << "Total: " << total << endl;`
Leer un número o una palabra	`cin >> variable;`
Leer una línea con espacios	`getline(cin, texto);`
`getline` después de `cin >>`	`getline(cin >> ws, texto);`
2 decimales (todo lo que sigue)	`cout << fixed << setprecision(2);`
Ancho fijo para el siguiente dato	`cout << setw(10) << dato;`

Operadores y reglas

- Precedencia: `( )` → unarios → `* / %` → `+ -` → `<<` → comparaciones → `&&` → `||` → `=`.
- Igual precedencia: de **izquierda a derecha** (excepto asignaciones y unarios).
- `int / int` = división entera. `%` = resto (solo enteros).
- División real con enteros: `static_cast<double>(a) / b`.
- `int x = 9.99;` guarda 9 (se corta, no se redondea).

Funciones de `string` (con `s = "Hola Mundo"`)

Función	Resultado
`s.length()`	`10`
`s[0]`	`'H'`
`s.find("Mundo")`	`5` (o `string::npos` si no está)
`s.substr(5, 5)`	`"Mundo"` (desde 5, toma 5)
`s.substr(5)`	`"Mundo"` (desde 5 hasta el final)
`s.replace(0, 4, "Chau")`	`s` queda `"Chau Mundo"`
Antes de la posición `p`	`s.substr(0, p)`
Después de la posición `p`	`s.substr(p + 1)`

Pseudocódigo ↔ C++

Pseudocódigo	C++
`Definir x Como Entero / Real / Cadena / Caracter / Logico`	`int / double / string / char / bool x;`
`Leer x`	`cin >> x;` o `getline(cin, x);`
`Escribir "texto", x`	`cout << "texto" << x;`
`x ← expresión`	`x = expresión;`

Diagrama de flujo

Óvalo = Inicio/Fin · Paralelogramo = Leer/Escribir · Rectángulo = Proceso · Rombo = Decisión · Flechas = orden.

UML → C++

`- atributo : tipo` → `private: tipo atributo;` · `+ operacion() : tipo` → `public: tipo operacion() { … }` · `void` = no devuelve nada.

20. Autoevaluación

Responde sin mirar y luego revisa las respuestas al final. Todas fueron comprobadas ejecutando el código.

1. ¿Qué muestra `cout << 17 / 5 << " " << 17 % 5;`?
2. ¿Qué valor guarda `double r = 9 / 2;`?
3. ¿Y `double r = 9 / 2.0;`?
4. ¿Cuánto vale `2 + 3 * 4 % 5`?
5. ¿Cuánto vale `20 / 4 / 2`?
6. Con `string s = "Lenguajes";`, ¿qué devuelve `s.length()`?
7. ¿Qué devuelve `s.substr(3, 4)`?
8. ¿Qué devuelve `s.find("aje")`?
9. Con `string t = "UNMSM-FIEE";`, escribe una instrucción que extraiga `"FIEE"` usando `find`.
10. ¿Qué tipo de dato usarías para un número de celular? ¿Por qué?
11. ¿Qué imprime `cout << (double)(7 / 2);`?
12. ¿Qué imprime `cout << static_cast<double>(7) / 2;`?
13. Si el usuario escribe `Ana Maria` y el programa hace `cin >> nombre;`, ¿qué se guarda?
14. ¿Por qué un `getline` que va después de `cin >> edad;` lee un texto vacío? ¿Cómo se soluciona?
15. ¿Qué muestra `cout << fixed << setprecision(2) << 3.14159;`?
16. En UML, ¿qué significa `- saldo : double`?
17. Escribe en C++ la declaración de la operación UML `+ calcularPromedio() : double`.
18. Encuentra los dos errores: `int main() { Cout << "Hola" }` (asumiendo que los `#include` y el `using namespace std;` están bien).
19. ¿Cómo queda `correo` después de `string correo = "ana@gmail.com"; correo.replace(4, 5, "outlook");`?
20. Con `string u = "Programa";`, ¿qué muestra `cout << u[0] << u[u.length() - 1];`?

Respuestas

1. `3 2` (17 / 5 = 3 con resto 2).
2. `4` (división entera: 9 / 2 = 4; se guarda 4.0).
3. `4.5`.
4. `4` → primero `3 * 4 = 12`, luego `12 % 5 = 2`, al final `2 + 2 = 4`.
5. `2` → de izquierda a derecha: `(20 / 4) / 2 = 5 / 2 = 2`.
6. `9`.
7. `"guaj"` (índices 3, 4, 5 y 6: g-u-a-j).
8. `5` (la `a` de «aje» está en el índice 5).
9. `t.substr(t.find("-") + 1)` → `"FIEE"`.
10. `string`: no se hacen cálculos con él y puede empezar con 0 o tener `+51`.
11. `3`: primero `7 / 2 = 3` (enteros) y luego se convierte a 3.0.
12. `3.5`.
13. Solo `"Ana"`: `cin >>` se detiene en el espacio.
14. Porque `cin >>` deja el Enter (`'\n'`) en el buffer y `getline` lo toma como una línea vacía. Solución: `getline(cin >> ws, texto);` o `cin.ignore();` antes del `getline`.
15. `3.14`.
16. Es un **atributo privado** llamado `saldo` de tipo `double`.
17. Dentro de la clase, en la sección `public:` → `double calcularPromedio();` (o con su cuerpo `{ … }`).
18. `Cout` debe escribirse `cout` (C++ distingue mayúsculas de minúsculas) y falta el `;` después de `"Hola"`. Además, en el curso siempre se termina `main` con `return 0;`.
19. `"ana@outlook.com"` (se borran 5 caracteres desde el índice 4, «gmail», y se pone «outlook»).
20. `Pa` (primer y último carácter).

21. Glosario

Término	Significado
Algoritmo	Secuencia finita, ordenada y precisa de pasos para resolver un problema
Asignación	Guardar un valor en una variable (`=` en C++, `←` en pseudocódigo)
Asociatividad	Regla que decide el orden entre operadores de igual precedencia
Biblioteca	Código ya hecho que se incluye con `#include` (`iostream` , `string` , `iomanip`)
Bit / byte	Bit: unidad mínima (0 o 1). Byte: 8 bits
Buffer de entrada	Zona donde espera lo que el usuario escribió hasta que el programa lo lee
Clase	Molde que define atributos y operaciones (UML / `class`)
Código fuente	El programa escrito en un lenguaje (archivo `.cpp`)
Comentario	Texto que el compilador ignora (`//` o `/* */`)
Compilador	Programa que traduce el código fuente a lenguaje máquina
Concatenar	Unir textos (`+` con `string`)
Constante	Valor que no cambia (`const double IGV = 0.18;`)
Constructor	Método especial que inicializa un objeto al crearlo
Declaración	Crear una variable indicando su tipo (`int edad;`)
Diagrama de flujo	Representación gráfica de un algoritmo
Enlazador (linker)	Une el código traducido con las bibliotecas y genera el ejecutable
Expresión	Combinación de valores y operadores que produce un resultado
Identificador	Nombre de una variable, función o clase
Índice	Posición de un carácter en un texto (empieza en 0)
Inicializar	Dar el primer valor a una variable
Literal	Valor escrito directamente en el código (`25` , `2.5` , `'A'` , `"Hola"`)
Método / operación	Función que pertenece a una clase
Objeto	Ejemplar concreto de una clase, con sus propios valores
Precedencia	Prioridad de un operador sobre otro
Programa ejecutable	Archivo en lenguaje máquina listo para ejecutarse (`.exe`)
Prueba de escritorio	Ejecutar un algoritmo a mano anotando los valores de las variables
Pseudocódigo	Algoritmo escrito en lenguaje natural estructurado
Semántica	El significado de las instrucciones
Sintaxis	Las reglas de escritura de un lenguaje
`static_cast`	Conversión explícita de tipo (`static_cast<double>(a)`)
UML	Lenguaje gráfico estándar para diseñar programas
Variable	Espacio de memoria con nombre, tipo y valor

¡Éxitos en el parcial! Si entendiste los 10 ejercicios de esta guía, entendiste los contenidos de las semanas 1 a 7.